GAPMAP

Design Document

Tayef Wasee – 33849950 & Ismaeel Khan 33819922

IS2018H: Computing Project 2

CONTENTS

DETAILS 3
CONCEPT INTRODUCTION 4
1.1 Problem statement 4
1.2 Proposed solution 5
1.3 Project novelty and value 6
1.4 Aims and objectives 8
1.5 Deliverable components 9
1.6 Scope and boundaries 9
STAKEHOLDERS, RESEARCH AND REQUIREMENTS GATHERING 12
2.1 Stakeholder groups 12
2.2 Context of use 14
2.3 Representative personas 16
2.4 Prior knowledge and domain analysis 16
2.5 Competitor and market comparison 17
2.6 Requirements elicitation strategy 20
2.7 User stories and acceptance criteria 23
PLANNING AND DEVELOPMENT METHODOLOGY 24
3.1 Chosen methodology 24
3.2 Proposed milestone timeline 26
3.3 GitHub workflow and evidence trail 26
3.4 Risk management and feasibility 28
PROTOTYPING AND ITERATIVE DESIGN 31
4.1 Low-fidelity concept 31
4.2 Medium-fidelity prototype 32
4.3 High-fidelity prototype 34
4.4 Design heuristics and accessibility 37
4.5 Planned iterative refinements 39
FORMAL SYSTEM DESIGN 40

5.1 Information architecture 40
5.2 Use case specification 42
5.3 Activity design 44
5.4 Sequence design 46
5.5 Technical architecture 47
5.6 Data design 49
5.7 API design overview 52
5.8 Recommendation engine design 53
5.9 Validation and state integrity rules 55
TECHNICAL SPECIFICATION 56
6.1 Front-end stack 56
6.2 Back-end stack 57
6.3 Database stack 58
6.4 Security and privacy design 59
6.5 Accessibility and responsiveness requirements 59
6.6 Maintainability and code organisation 61
EVALUATION AND TESTING PLAN 64
7.1 Evaluation goals 64
7.2 Formative evaluation 65
7.3 Functional and validation test strategy 67
7.4 Usability and accessibility test plan 67
7.5 Acceptance criteria 70
ETHICAL, LEGAL AND PROFESSIONAL CONSIDERATIONS 71
CONCLUSION 73
REFERENCES 75
APPENDIX A. FULL FUNCTIONAL REQUIREMENT SET 76
APPENDIX B. FULL NON-FUNCTIONAL REQUIREMENT SET 77
APPENDIX C. SAMPLE FORMATIVE INTERVIEW QUESTIONS 78
APPENDIX D. DIAGRAM SET INCLUDED IN THIS DOCUMENT 79

DETAILS

Student(s)	Tayef Wasee – 33849950 – twase001@gold.ac.uk Ismaeel Khan – 33819922 – ikhan004@gold.ac.uk
Module	Computing Project 2
Tutor	Sean McGrath
GitHub Repository	github.com/Tayef006/GapMap
Technology Stack	React, Vite, Node.js, Express and SQLite

CONCEPT INTRODUCTION

1.1 Problem statement

University students frequently navigate fragmented daily schedules influenced by a combination of academic, personal, and professional obligations. A normal day might include lectures, seminars, lab work, moving between campus buildings, commuting, working part-time, hanging out with friends, and studying on your own. Because of this, students don't often get long stretches of free time. Instead, they usually have short breaks between planned activities throughout the day. Even though these breaks may not seem important, they can add up to a lot of time that could be used productively.

However, it's not always easy to make the most of these short times. In real life, students may have a hard time figuring out what to do during a 20-, 40-, or 60-minute break. Some tasks may need more focus, planning, or time than is available, while others may be fine but not obvious to the user right away. This uncertainty often makes people unable to make decisions, switch contexts quickly, or do things that aren't very useful, like checking social media or browsing on their phones. So, time that could be used to make progress that is both manageable and meaningful is often wasted.

This makes the design task harder than just storing tasks or showing a schedule. It's not just a matter of whether users know what they need to do or whether they can see when they're busy. It's more about whether a system can link those two types of information in a way that helps people make quick, smart, and reliable decisions. For a system to be truly useful, it must assist the user in addressing an immediate practical inquiry: what is the most feasible and valuable task I can accomplish within the present time constraints?

This problem is only partially solved by current productivity tools. Generic to-do list apps are good for keeping track of tasks, deadlines, priorities, and categories, but they usually don't help you plan when to do things. They might show what needs to be done, but they don't usually check to see if a task is suitable for the user's current free time. Calendar and timetable tools, on the other hand, are good for showing fixed commitments and open slots, but they usually don't show how those slots fit into a task list. That is to say, one type of system keeps track of obligations and the other shows availability, but neither of them directly helps with the decision-making process that happens between the two.

GapMap is meant to fill this specific gap by having a clear and valuable system purpose. It doesn't want to replace all the productivity software that already exists. Instead, it wants to do one thing very well: find real gaps in your schedule and suggest tasks that fit those gaps. The system tries to help students make better use of their free time by combining schedule data with task information and using clear recommendation rules. This narrow scope is intentional because it lets the application focus on a specific user need while still being possible to design, build, and test within the project's limits.

1.2 Proposed solution

GapMap is a proposed web-based productivity tool that would help students make better use of the free time they have between scheduled activities. There are five main pages in the system: Home, Dashboard, Timetable, Tasks, and Recommendations. Each page has a different job to do in the app's overall workflow. This makes it easy for users to go from entering information to getting useful, timely help.

The Home page is the first page you see when you open the system, and it tells you what the app is for. The Dashboard is a central place that shows a summary of the user's tasks, upcoming deadlines, and the best task to do right now. This lets the user quickly see where they are without having to go through a lot of different parts. The Timetable page lets users write down things like lectures, seminars, lab sessions, work shifts, or any other scheduled events that they have to do. Users can make and keep track of tasks that need to be done on the Tasks page. Finally, the Recommendations page shows tasks that are best for the user's free time, based on the information that has been entered in other parts of the system.

Utilising structured task and timetable data is an important part of the suggested solution. GapMap doesn't let users store tasks as simple text entries. Instead, it needs metadata that lets it make recommendations. For tasks, this includes things like how long you think it will take, how important it is, when it is due, and how much energy you have. These features help the system tell the difference between tasks that need to be done right away, tasks that can be done in a short amount of time, and tasks that need more focus or effort. In the same way, timetable sessions are stored in a structured way so that the system can accurately analyse start and end times and find real free periods between two commitments.

After you enter this information, the system looks at the timetable to find real gaps between planned sessions. This is an important feature because the app's value isn't just in storing information; it's also in how it helps people make decisions by interpreting that information. GapMap does this comparison automatically instead of making the user do it by hand. It finds out how long each gap is and then decides which tasks that are still open are best for that time.

A rules-based recommendation engine takes care of this process. The recommendation engine ranks tasks that aren't finished yet by looking at things like how well the task fits the time available, how important it is compared to its deadline, and how much effort or energy it takes. The system then gives the best task as the main recommendation, along with one or more alternatives if they are relevant. This method gives the user both help and freedom. The best option gives a clear suggestion for what to do right away, while the other options keep the user's choice and make them trust the system more.

The dashboard makes this feature even better by showing the recommendation in context. The dashboard shows the best available suggestion as part of the main interface, so the user doesn't have to ask for one every time. This helps people make decisions quickly and makes everyday use easier. So, the user can not only store and organise tasks, but they can also get help in a timely manner with what to do next.

1.3 Project novelty and value

The novel idea about GapMap is not that it can store tasks or show scheduled events, since these features are already common in productivity software. The project's originality comes from the fact that it sees "dead time" as a main design problem instead of a minor annoyance. In a lot of traditional systems, users can see short free periods between commitments, but the system doesn't actively interpret or support them. GapMap is different because it focuses on these broken-up times when people are available and tries to turn them into real chances to do useful work.

This framing makes the project stand out more than a generic study planner or task-tracking app. GapMap is not just a tool for keeping track of what you need to do; it also helps you figure out what you can realistically do in a certain amount of time. This changes the focus of the system from organising things passively to helping people make decisions actively. Because of this, the app deals with a more specific

and possibly more important productivity issue, one that is especially important for college students whose schedules are often broken, irregular, and interrupted.

This method has both practical and technical benefits. The system's goal is to help users save time, avoid making decisions, and choose the right tasks by giving them suggestions based on real availability instead of just the importance of the tasks. The system can help people use their time more effectively and make quick, confident decisions by getting rid of unrealistic choices and highlighting tasks that really fit the available gap. This gives GapMap a clear real-world use and helps explain why it was made instead of just being a simpler version of an existing productivity tool.

From a technical perspective, the project is also valuable because it introduces a more interesting design and implementation challenge than a standard CRUD-based task manager. The system needs to think about time windows, compare the length of tasks to actual gaps in the schedule, turn down bad suggestions, and explain its reasoning in a way that is easy to understand. This necessitates the amalgamation of various technical layers, encompassing a responsive frontend interface, an API for data interchange, a database for enduring storage, and business logic proficient in assessing and prioritising tasks based on established criteria. The interaction between these layers makes the project more complicated in a way that matters because the quality of the final system depends on how well they all work together to support the recommendation process, not just on how well each layer works on its own.

GapMap is an appropriately ambitious project for a second-year computing module. It has decision-making logic and a more specific problem focus, which makes it more than just a basic web app. However, it can still be done in the time and space of a semester-long project. Because of this, the project strikes a good balance between being original, technically sound, and doable. It is complicated enough to show that you can think like a designer, develop full-stack, and solve problems in a structured way, but it is still possible to plan, carry out, test, and evaluate within the time frame.

1.4 Aims and objectives

Area	Objective	Success indicator
Product goal	Design a usable system for turning fragmented time into productive task suggestions.	Users can add tasks and sessions, then receive understandable recommendations for real gaps.
Interaction goal	Keep the interface simple and low-friction.	Primary actions are discoverable without specialist instruction.
Technical goal	Implement a complete frontend-backend-database workflow.	Data persists, routes respond correctly, and pages refresh reliably.
Validation goal	Reject invalid data and protect user trust.	Past deadlines and invalid timetable values are prevented at more than one layer.
Academic goal	Evidence systematic planning, prototyping, and evaluation.	The report contains formal diagrams, milestones, prototype rationale, and testing plans.

1.5 Deliverable components

Deliverable	Description	Why it matters
Frontend application	React/Vite interface with page-based navigation and reusable components.	Provides the visible system and supports usability evaluation.
Backend API	Node/Express routes, controllers, and services.	Separates presentation from validation and business logic.
Database layer	SQLite schema and query models.	Supports persistent storage and enforceable constraints.
Recommendation service	Rules-based gap detection and task ranking logic.	Represents the project's distinctive technical challenge.
Prototype set	Low-, medium-, and high-fidelity representations.	Shows iterative design reasoning before and during build decisions .
Project evidence	Timeline, GitHub workflow, risk plan, and evaluation plan.	Demonstrates process maturity and academic rigour.

1.6 Scope and boundaries

For the GapMap project to stay possible, logical, and in line with its main goal, the scope must be clearly defined. The system's scope has been limited on purpose to only include the features needed to help users make better use of their free time between classes. The project can focus on technical quality, usability, and completeness by limiting the number of well-integrated functions instead of trying to add too many features.

The project's scope only includes features that are directly related to how the system works. These include making tasks, deleting tasks, and managing task status, which lets users add, remove, and update work items as their workload changes. The system also lets users create and delete timetable sessions, which lets them keep track of fixed commitments and keep their schedule up to date. There is also deadline validation to make sure that the task data the user enters makes sense and is consistent. This lowers the chances of getting false or misleading recommendations.

The implementation of recommendation generation is a key part of the project scope and is what makes GapMap stand out from other projects. The system will look at the stored schedule and task data to find real free times and suggest tasks that fit those times the best. Dashboard summaries back this up by giving the user a quick look at their workload, tasks that are due, and the most relevant current suggestion. Persistent storage is also included to make sure that data is always available between sessions. This lets tasks and timetable entries be stored safely in a database instead of just in memory for a short time.

The scope also includes several supporting features that strengthen the usefulness and credibility of the recommendation process. One of these is a user-defined active day window, which allows the system to distinguish between realistic working hours and irrelevant gaps outside the period the user considers part of their day. This makes suggestions more aware of the situation and keeps them from suggesting tasks at times when the user wouldn't normally consider them available. Recommendation explanations are another feature that helps make things clear by showing why a certain task was suggested. This is important for user trust because a recommendation system only works if its choices are clear and make sense to the user. Lastly, the project scope includes a structured evaluation plan that will let the system be evaluated systematically against its intended goals, usability goals, and technical requirements.

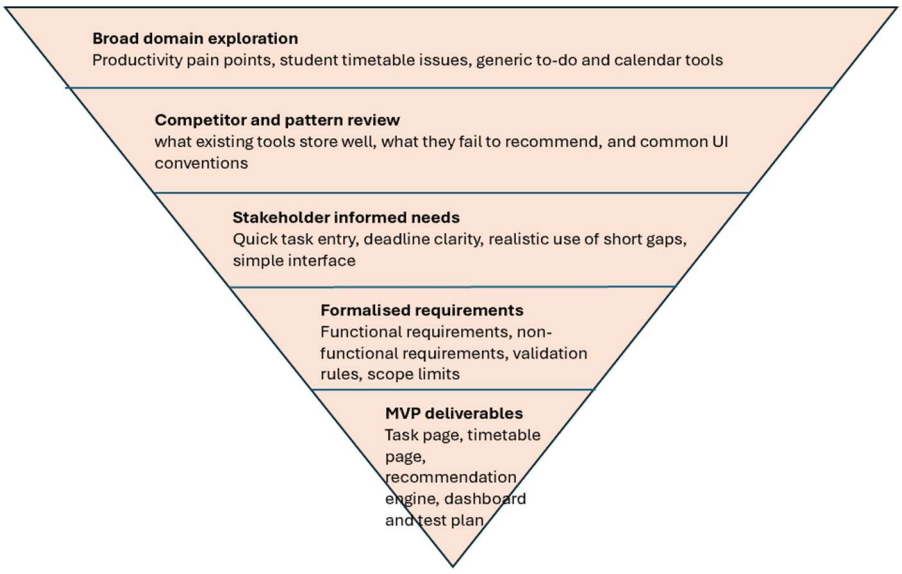
It is just as important to figure out what is not part of the minimum viable product. Multi-user authentication, social collaboration, external calendar synchronisation, push notifications, advanced analytics, machine-learning-based ranking, and enterprise deployment are some of the features that are not included in the MVP. These features might be useful in a bigger or commercial product, but they aren't necessary to prove the main idea of GapMap within the time frame of this project.

These exclusions are not mistakes; they are planned. They show that the project boundary is trying to stay realistic and strict. If you add too many secondary features, the app could become a broad but shallow collection of ideas that aren't fully

implemented, which would lower the overall quality of the final product. The project can stay focused on building a technically sound and user-friendly core system by not including more complicated features like machine learning, third-party integrations, and multi-user functionality. This method favours depth over breadth, which lets the project show off careful design, solid implementation, and useful evaluation instead of just covering a lot of features.

Figure A. Requirements Funnel

How broad research is narrowed into concrete requirements and MVP choices



STAKEHOLDERS, RESEARCH AND REQUIREMENTS GATHERING

To be successful, a software project needs to have a clear idea of who its users are, what environment it will work in, and what needs will come up in that environment. This is especially important for GapMap because the system's value depends not only on how well it works technically, but also on whether it solves a real and ongoing user problem. Because of this, the project needs to carefully think about stakeholder groups, relevant domain research, and ways to gather requirements to make sure that the proposed solution is both useful and realistic.

Stakeholder analysis is a structured way to figure out who is affected by the system, who has an impact on its development, and whose needs should come first when designing and evaluating it. The most important people for GapMap are the ones who are directly affected by the problem the system is meant to solve. But there are also secondary and longer-term stakeholders whose interests affect how the project should be shown, carried out, and evaluated. Identifying these groups helps set the project's priorities and makes sure that requirements are not set without considering real use cases or the project's overall goals.

2.1 Stakeholder groups

The student user is the main stakeholder because they have to deal with broken schedules and tight deadlines. This group includes people who commute, students who have to balance classes with part-time jobs, and students who often know what they should do but have trouble figuring out what fits into their schedule right now. Secondary stakeholders include tutors and assessors who need to see a credible design and development process, as well as future employers or portfolio reviewers who might look at the project as proof of technical skill. If the project grows beyond local development, long-term stakeholders could include future deployment hosts or service providers.

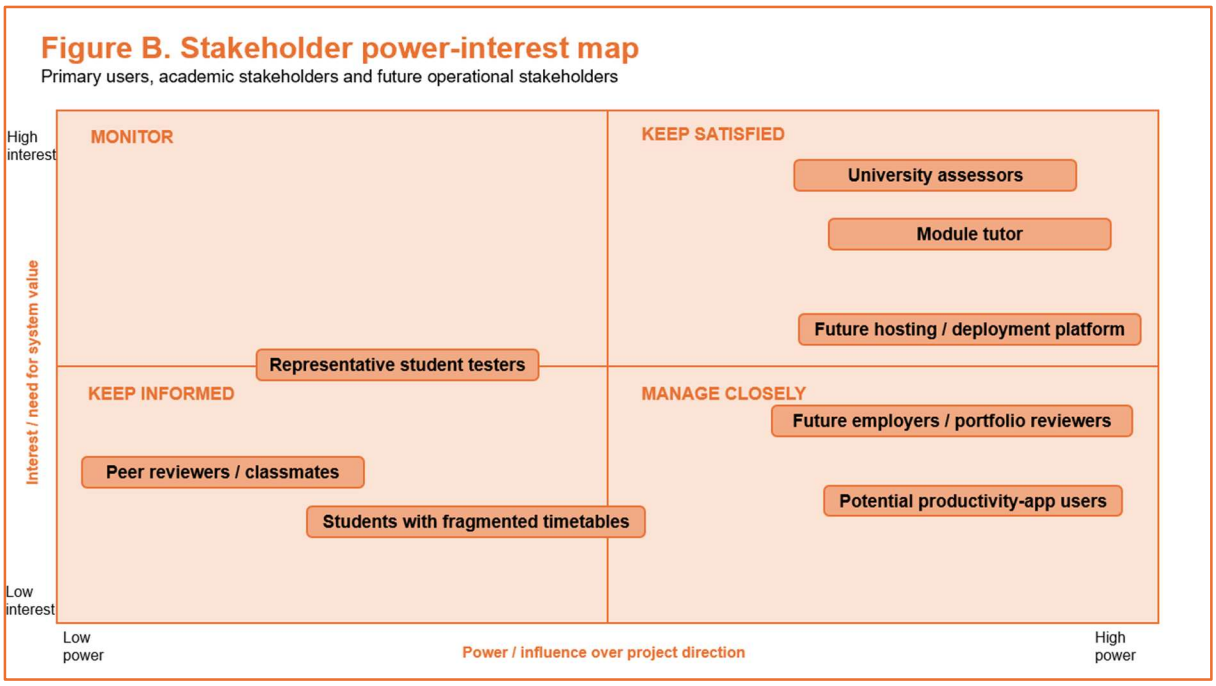
There are several subgroups within this main group of stakeholders whose needs help make the system's requirements clearer. For instance, students who have long commutes may have several short breaks during the day that could be used productively if the right tasks were found. Students who work part-time while going to

school may need quick recommendations that take into account how tired they are and how little time they have. Other students may be academically organised by writing down deadlines and tasks, but they may still have trouble acting quickly because they don't have anyone to help them figure out how to fit those tasks into their real schedules. These subgroups make the problem seem more important and help explain why the project is focused on it.

The secondary stakeholders are the tutors and assessors who work with the academic module. They aren't the same kind of end users of the application as students, but they are still important because they judge how well the project works as a piece of computing work. They want to see proof of a reliable design and development process, which includes clearly defining the problem, analysing the requirements, implementing the solution, testing it, iterating on it, and critically evaluating it. Because of this, their role affects the level of documentation, justification, and rigour needed for the whole project. So, the system needs to be built so that it works for users and shows good software engineering.

Another group of secondary stakeholders is future employers, recruiters, or portfolio reviewers who may later see the project as proof of technical skill. From this point of view, GapMap is not just a piece of software; it also shows that the developer is good at full-stack web development, data modelling, validation, designing business logic, and solving problems from the user's point of view. This group of stakeholders has an indirect effect on the project because it pushes for a high level of implementation and documentation. In this case, a system that is well-defined and well-implemented is more useful than a project that is too big and has features that aren't fully explained or justified.

If the project were ever to grow beyond a local academic prototype, a second group of long-term stakeholders would be potential deployment hosts, service providers, or maintainers. These stakeholders are not currently central to day-to-day development, but they are still important to think about when it comes to things like scalability, maintainability, hosting needs, and future growth. Including them helps show that people are aware that software systems exist in larger technical environments and may eventually need help outside of the original development context.



2.2 Context of use

It's important to know the context in which GapMap is supposed to be used because the system's effectiveness depends not only on its features but also on how well those features fit the user's real-world work habits and limitations. People should mostly use GapMap in short, focused bursts on a laptop or desktop browser. This is probably how student users will act. They might use the app for a short time to plan their week, update their task information after getting a new deadline, or see what work they can do right before a free period starts.

This context of use has a direct effect on how the system is designed. Because users probably won't spend a lot of time exploring the app without stopping, the interface needs to allow for quick and easy interaction. Without having to go through extra steps, key actions should be easy to find, and the main screens should make it clear what the most important information is. The recommendation function, which is the most important part of the system, should not be hidden behind too many menus or require a long, complicated process to get to. The design should instead make it easy for the user to quickly get useful recommendations with little effort.

The context of use also implies that the application must be responsive to urgent needs. A user can open GapMap right before a lecture, right after a class, or in between other things they have to do. In these cases, the system needs to show a clear and up-to-date list of tasks that are available and time slots that are open

without requiring a lot of work from the user. This means that the interface should put clarity, quick loading, and easy access to summaries and recommendations at the top of the list. Any delay, confusion, or unnecessary interaction would make the app less useful in the situations it was made to help with.

Another important thing to think about is how long the item will be used. GapMap needs to be able to quickly recover from interruptions because users are likely to enter and leave the app often, sometimes in the middle of other tasks. So, data persistence is very important. Users shouldn't have to keep making the same information over and over again, so tasks, timetable sessions, and user preferences should stay available between sessions. In the same way, important state should be remembered when needed to make repeated use easier. For instance, the system should remember and use preferences like the user-defined active day window, so that it stays personalised and works right away without having to be set up again.

This context also makes it clear how important it is to keep the overall interaction model simple and easy to understand. Users who use the app between tasks probably won't be able to handle a steep learning curve or workflows that are too complicated. So, the design should have a clear structure, navigation patterns that are easy to understand, and actions that are easy to see. The system should be able to handle both short-term planning, like looking over the next week, and quick, in-the-moment interactions, like checking what task fits the next available gap. Designing for both of these situations makes the app more useful and makes sure it stays useful at different times in the user's day.

2.3 Representative personas

Persona	Profile	Design Implications
Commuter student	Travels to campus several days a week and has many 20–50-minute gaps between sessions.	Needs very fast task entry and realistic short-gap recommendations.
Deadline-driven student	Often tracks tasks but underestimates urgency until deadlines are near.	Needs visible deadlines and ranking rules that push urgent work upwards.
Low-friction organiser	Dislikes cluttered productivity software and wants only the essentials.	Needs simple page structure, obvious labels and clear feedback.

2.4 Prior knowledge and domain analysis

GapMap is in the middle of productivity software, timetable planning, and lightweight decision-support systems. It's important to understand this bigger picture because the project isn't happening in a vacuum. It builds on what we already know about how people manage tasks, organise their time, and use digital tools that are meant to help them be more productive. A review of this field shows that many current systems are good at storing information, but not as good at helping users make quick and useful decisions when they don't have a lot of time.

A common problem in the field of productivity is that software can fail when it requires too much planning. If tools that are supposed to help with organization require a lot of setup, regular manual updates, or complicated categorisation before they can be useful, they may actually make things worse. In these situations, the work needed to keep the system running can be more trouble than it's worth. This is especially important for students, who often have to balance schoolwork with other responsibilities and are less likely to use systems that seem like a lot of work or take a lot of time. In this case, a productivity app needs to make things easier for the brain, not harder.

This is also true when you look at human-centered design principles. Guidance in this area says that digital systems should help users by making decisions easier, lowering friction, and showing information in a way that is clear and easy to understand. One common design risk is giving users too many choices, making things too complicated to look at, or giving them feedback that is hard to understand. Users may feel tired of making decisions, confused, or less trusting of the system when this happens. This is especially important for GapMap because the app is meant to help people make quick decisions when there are short time gaps. The system would not work if the interface or the process for making recommendations was hard to understand.

This analysis of the domain directly affects the design of GapMap. The project is purposefully limited in scope so that it doesn't try to compete with full-fledged calendar platforms, task ecosystems, or highly customisable productivity suites. It doesn't want to replace every tool the user already uses; it just wants to solve one specific problem that isn't getting enough attention. The problem is how to quickly, clearly, and usefully match pending tasks to realistic gaps in the schedule. The system can stay light, focused, and useful by only focusing on this one function.

From a design and development point of view, this narrow positioning is also good. It lets the project focus on depth over breadth, making sure that the recommendation feature is implemented in a way that makes sense instead of being surrounded by a lot of less developed secondary features. The domain research thus validates the choice to characterise GapMap as a specialised decision-support tool rather than a versatile productivity platform. This makes the project more distinct, strengthens its usability goals, and makes it more likely that the final system will be useful to the people who will use it.

2.5 Competitor and market comparison

When doing competitor analysis for GapMap, it's better to look at product categories than just brand-to-brand competition. This is because the project doesn't want to compete with one specific application in a business sense. Instead, it is part of a larger group of digital productivity tools, each of which solves a different part of the problem that GapMap is meant to solve. Looking at these categories makes it easier to see both the gap in the market and how the proposed system is meant to fit in.

The basic to-do app is the first relevant category. Users can make tasks, set due dates, add labels or priorities, and organise their work into lists or projects in a

flexible environment that these systems usually offer. Their strength is in how simple and flexible they are. Users can often organise information in a way that works best for them, which makes these tools great for keeping track of tasks and progress. But their biggest problem is that they usually don't give the user much information about their real schedule. They usually say what needs to be done, but they don't usually say when the user is free to do it. This means that the user is still responsible for matching tasks to their available time.

The second type is a digital calendar or timetable app. These systems are good at giving time context. They let users see their scheduled commitments, find out when they are busy, and see their day or week in a structured time-based way. This makes them useful for figuring out when people are free and planning around events that are already set. But they usually don't do a good job of helping with action selection. A calendar might show that a user has a free 40-minute block of time, but it usually doesn't help them figure out what task would be best for that time. In this way, calendars are good at showing time, but they don't show how that time relates to a list of tasks.

The study planner or academic productivity app is another important category. These tools usually fall somewhere between task managers and calendars because they let you keep track of assignments, plan revisions, and keep an eye on deadlines. They may give students more academic structure than a regular to-do list and are often better suited to how students use them. But a lot of these systems are still focused on subjects instead of gaps. They often focus on organising work by module, exam, or project instead of helping the user make the most of small, broken-up chunks of time throughout the day. So, even though they might make it easier to see how much work needs to be done, they don't always solve the specific problem of making decisions in the moment that GapMap is trying to solve.

This comparison shows a big hole in the current market. Most productivity tools today do one of two things well: they either keep track of what the user needs to do or show when the user is busy. There aren't many systems that directly connect these two areas in a simple and useful way. In particular, it seems that not much is being done to help users figure out what realistic task to do in a short, specific free time between scheduled sessions. This gap is what GapMap is based on and shows that it is a separate project that is still useful.

This analysis helps to better define GapMap's intended market position. It is not meant to be a general-purpose planner, a full replacement for an external calendar, or a broad academic management platform. Instead, it is presented as a task recommender that knows about schedules. It stands out because it can combine

knowledge of schedules with task metadata to make suggestions that take into account the length and usefulness of real free time gaps. This focused positioning gives the system a clearer identity and keeps it from having to compete with bigger, more feature-rich platforms on their own terms.

Tool category	Typical strength	Typical weakness	GapMap design response
Generic to-do apps	Fast task capture and simple lists	Do not connect tasks to actual free timetable windows	Store task metadata, then rank tasks against specific gaps
Digital Calendars	Strong visibility of occupied and free time	Do not decide what work best fits that time	Treat calendars as input context rather than the whole solution
Study Planners	Useful for students and subject organisation	Can become cluttered and still rely on manual planning	Focus on narrow decision support instead of broad life management
Workspaces / note systems	Highly flexible and customisable	Flexibility can increase setup overhead and inconsistency	Provide a clear and default structure and constrained data entry

2.6 Requirements elicitation strategy

The strategy for getting GapMap's requirements should be based on a funnel, going from a broad investigation to a precise system definition. This method works because the project starts with a broad productivity issue and needs to narrow it down to a set of clear, actionable, and testable software requirements over time. At the broadest level, research is first used to learn more about the bigger problem area, such as how students handle their workload, how broken schedules affect productivity, and how current tools do or don't help with this issue. This broad understanding is the basis for the project and helps make sure that the proposed system is based on a real user need and not just an assumption.

From this point on, the elicitation process should get more focused on requirements that are based on what stakeholders want. Once you know what the bigger problem is and how the market works, you can focus on what the intended users need, how

they act, and what frustrates them. For GapMap, this mostly means student users who are under a lot of pressure to meet deadlines, have busy schedules, and have trouble figuring out how to make the most of their free time. After that, these needs can be turned into user stories, which help show requirements from the end user's point of view, and then into formal functional and non-functional specifications. This way, the process of getting information goes in a structured order from general ideas to specific system expectations.

This method follows the project guidance given for the module, which clearly says to move from a general understanding of the domain to more specific user needs. The exemplar reports also back it up by showing a similar process that combines market research, stakeholder consideration, prototyping, and formal specification into a single evidence trail. Following this structure makes the project more credible because it shows that the requirements weren't just picked at random, but were instead the result of a logical and traceable design process. This is important for both software engineering and meeting the academic standards of the test.

There are many practical ways to get people to talk about a project of this size. One helpful way to do this is to have short, semi-structured interviews with other students. These can give you direct qualitative information about how students are currently managing their tasks, what problems they have with broken-up time, and what features they would find useful in a system like GapMap. The interviews are semi-structured, which means that they can be consistent across participants while still allowing for new issues or priorities to come up.

Another good way to do this is to watch how people plan right now. This could mean casually watching how students use calendars, to-do lists, reminders, or study planners in their daily lives. Observation can uncover practical behaviours that users may not consistently articulate during interviews, including dependence on specific applications, patterns of underutilisation, or inefficiencies in transitioning between tools. This is especially useful for GapMap because the project is about making decisions in the moment, not just planning for the future.

A third way is to use short online surveys that ask about planning habits, managing deadlines, and how people feel about how they use their free time. Surveys are a good way to get answers from a larger group of people in a short amount of time. They can also give simple numbers to back up patterns found in interviews or observations. A survey could help find out how often students waste short breaks between classes, how often they use calendars and task managers, or how sure they are about what to do when they have some free time.

You can also use paper-based or digital wireframe walkthroughs to get early feedback on proposed interface ideas and workflows. This method is especially helpful in the later stages of elicitation because it lets potential users respond to both the idea of the system and how it is planned to work and interact. People can say if they think the layout is clear, if the navigation is easy to use, and if the recommendation process seems trustworthy and helpful. Testing assumptions before the implementation is done helps to further refine the requirements.

These methods are intentionally light, which makes them fit the size and time frame of the project. GapMap isn't meant to be used for big industrial research projects, but it still needs enough proof to back up its design choices and show that its requirements are based on real user needs. The project can create a set of requirements that is both manageable and well-supported by combining broad domain research with input from key stakeholders and feedback on prototypes.

2.7 User stories and acceptance criteria

ID	User Story	Acceptance Criterion
U1	As a user, I want to add a task with duration and deadline, so that the system can recommend it later.	A valid task is stored and displayed immediately after submission.
U2	As a user, I want to add timetable sessions so that free time can be calculated.	Sessions are stored with day and times and shown back in order.
U3	As a user, I want the system to recommend tasks only for real gaps between sessions.	No artificial start-of-day or end-of-day gap is returned.
U4	As a user, I want deadlines earlier than today to be rejected.	Past dates are blocked in the UI and rejected by the backend.
U5	As a user, I want to understand why a task was suggested.	Recommendations include a readable rationale and alternatives where available.

PLANNING AND DEVELOPMENT METHODOLOGY

3.1 Chosen methodology

The best way to develop GapMap is to use an iterative, agile-style method along with the ideas of user-centred design. This combination works well for the project because both the recommendation logic and the user interface will likely get better with each iteration during development. From the start, the project doesn't have a set of outputs that are completely fixed and predictable. Instead, it includes design choices that can be improved by testing, feedback, and small changes over time. Because of this, a strict linear approach would not be as good as one that lets the system change slowly over time.

A strict Waterfall approach, in which each stage is completed fully before the next begins, would be less suitable for this project. Waterfall can help with structure and documentation, but it's not as good when important parts of the system need to be improved because of problems with implementation or feedback from users. In GapMap, you may need to change your mind about the recommendation engine's exact shape, how clear its explanations are, and how easy it is to use. If development followed a strict step-by-step plan, it would be harder to change these choices after implementation had begun. This would make it more likely that a technically complete system would be made that doesn't fully meet user needs or give the best experience.

Agile-style development, on the other hand, allows for incremental delivery, continuous testing, and the ability to make changes as the project moves forward. This is especially useful for GapMap because it lets them build the app in small steps, with each step adding a new layer of functionality to the system. With an iterative workflow, you can test core features early, find design flaws, and improve implementation before the system gets too complicated. It also helps with better risk management because problems can be found in smaller parts of the project instead of just at the end.

User-centred design is also important because GapMap's success depends a lot on how easy it is to use and how useful it is in real life. The system is meant to help students act like real students when they don't have much time, so decisions about the interface should be based on how users actually plan to work and respond to suggestions, not just what the developers think. A user-centred approach helps make sure that navigation is clear, that the recommendations are easy to

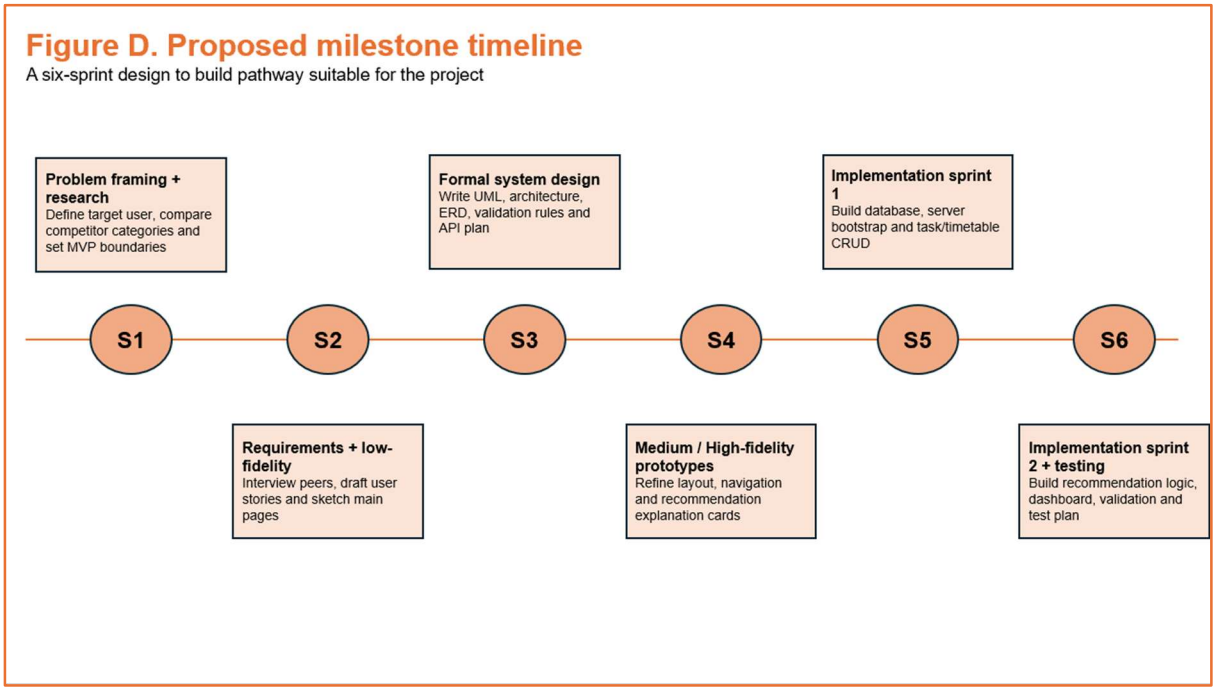
understand, and that the overall structure of the system is based on real-world use cases. This is especially important for a project like GapMap, where value depends not only on how technically correct the recommendations are, but also on whether the user thinks they make sense and can be trusted.

Feature slicing is the best way to use this method in real life. Instead of trying to build the whole system at once, development should go in order of logical dependencies and add small but complete slices of functionality. For GapMap, this means that the timetable layer needs to be made before the recommendation engine. This is because recommendations need accurate timetable data to find real free periods. Once the timetable feature is stable, the task layer can be added. This will let users make, organise, and save tasks with the structured metadata they need to rank them later.

After these two basic layers are in place, the next step should be to make recommendations. This is where the system's rules-based logic combines gaps in the schedule with task attributes. Only after this core mechanism is working well should work on dashboard aggregation, summary views, and other interface improvements begin. This order is good because it makes sure that the system's most important logic is based on data structures that are stable. It also makes testing and debugging easier because you can check each stage before adding the next dependency.

This order lowers the risk of the project in a number of ways. First, it stops people from spending time making the interface look nice before the core logic is working. Second, it makes it easier to find problems with implementation because each layer can be tested on its own and then with the next layer. Third, it encourages steady visible progress because each slice that is finished goes directly into a working version of the application. This is in line with agile thinking, which says that progress should be measured by working increments instead of just big end-stage goals.

3.2 Proposed milestone timeline



3.3 GitHub workflow and evidence trail

Version control in the GapMap project should not just be a way to store code. It is also an important part of the documented design and development process because it shows how the system changed over time. In this module, project management evidence is an important part of the assessment. The project brief and the example report both make it clear that showing structured progress through tools like Git branches, Kanban boards, Gantt charts, milestone logs, and other forms of traceable development activity is important. This is why GapMap should use GitHub to support not only technical implementation, but also the communication of a disciplined and well-managed workflow.

A well-kept repository can show how the project went from ideas to a working system. Instead of showing development as one final upload, version control should show important steps along the way. Commits should show how important features were added, how the structure was changed, how bugs were fixed, how improvements were made to testing, and how the development process was made clearer when reviewed later. This is important because it shows that the project was put together in a planned and gradual way, not in a disorganised or rushed way.

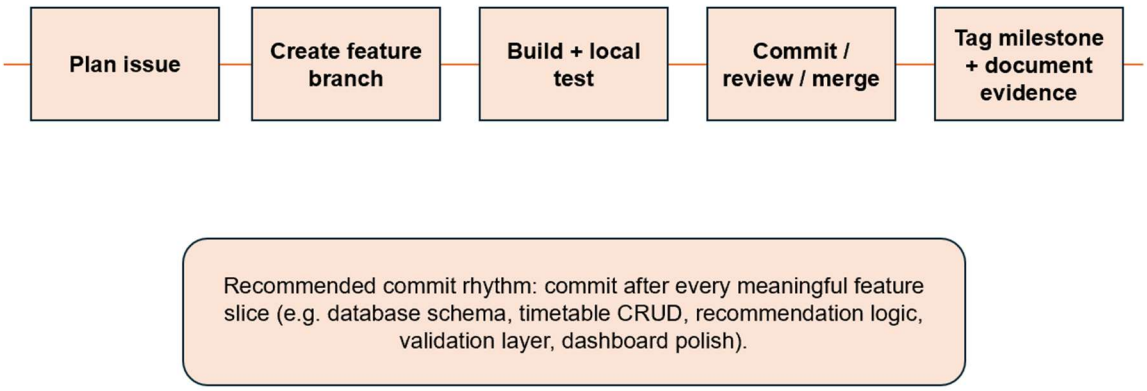
For GapMap, this means that the repository should keep track of clear progress points, like when the timetable feature was added, the task model was put into use, the recommendation logic was added, the validation was improved, and the dashboard was integrated. The commit history should show each of these changes as a big step in the system's growth. So, commit messages should be clear and useful so that a reviewer can see what was changed and why. A repository with a series of small, clear changes is much better proof of design thinking and technical progress than a single large code dump at the end.

Using GitHub also makes it easier to manage projects while they are being worked on. Changes can be looked at in relation to milestones or planned goals, and features can be made in a more controlled way. Using a structured workflow can help lower risk even in an academic project with only one developer. It does this by making it easier to find problems, undo changes if needed, and keep track of when certain features were added or changed. This is especially helpful for a project like GapMap, where bugs can happen when frontend components, backend routes, database structures, and recommendation logic interact. Keeping a clear record of development history makes it easier to find and fix these bugs.

The repository may also be supported by other project management tools, like a simple Kanban board, a list of milestones, or logged development checkpoints, if they are needed. These don't have to be big, but they make the evidence stronger by showing that the work was planned and watched over instead of being done on the fly. This makes the overall submission stronger in the context of the module because it connects technical output with visible process management.

Figure E. Proposed GitHub workflow

Version control supports safe iteration, evidence of progress and rollback



3.4 Risk management and feasibility

A good design document should critically look at how likely it is that the proposed system can be built, rather than just assuming that it will work. This is an important part of planning software because even a good idea can fail if its scope, technical requirements, or usability problems aren't handled properly. For GapMap to be possible, the system must be able to provide a useful recommendation feature within the project's time, budget, and technical limits. Because of this, risk management needs to be a clear part of the design process.

Overscoping is one of the biggest risks that come with GapMap. It is possible for productivity software to grow into many other areas, which could make the project too big if it tries to add too many extra features on top of its main function. User accounts, synchronisation, notifications, analytics, and collaboration are all features that might seem useful, but adding them too soon would make the system more complicated and less likely to work well. When you have a lot of time to work on an academic project, overscoping is especially dangerous because it can lead to a lot of parts that are only partially finished instead of a few that are well-implemented.

A second big risk is that the recommendations aren't very good. GapMap is all about suggesting tasks that fit real gaps in the schedule. This means that the app's usefulness depends a lot on whether the user thinks these suggestions make sense. If the recommendation engine makes suggestions that don't fit with the time available, don't seem urgent, or don't make sense in the context, the system is much less useful. Bad recommendations would not only make the project less useful, but they would also go against the project's main design claim that task support that is aware of the schedule can boost productivity.

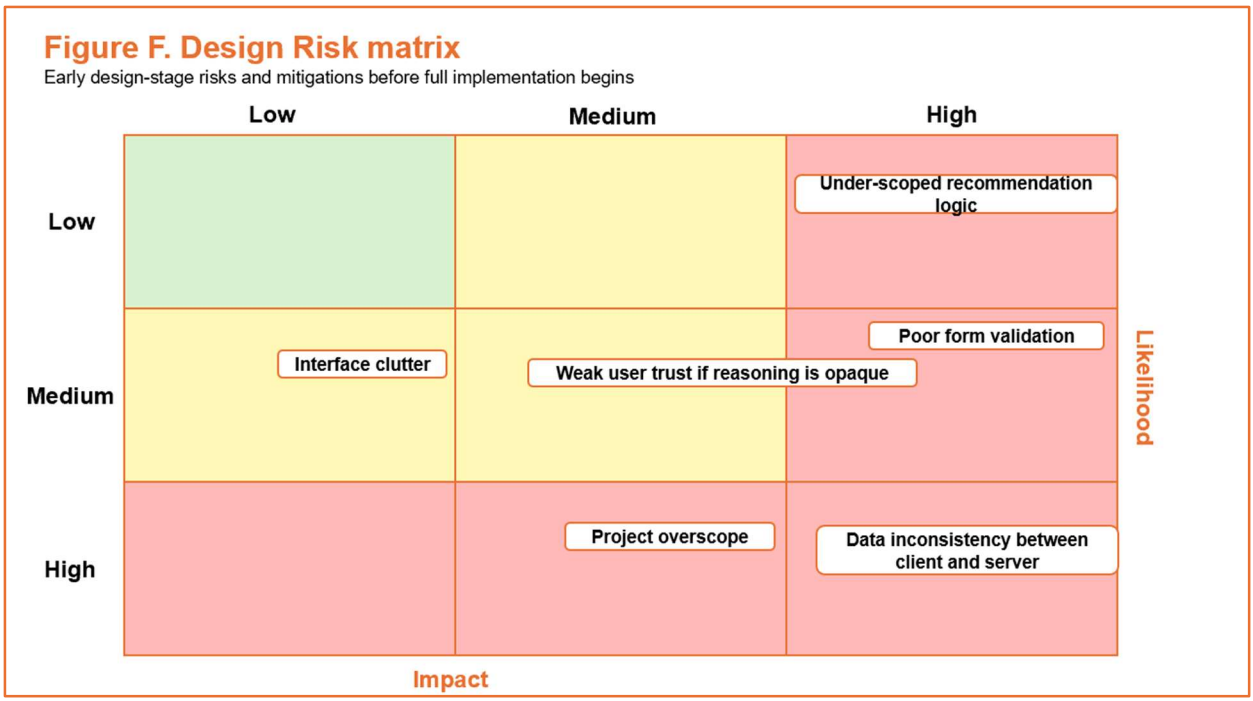
Another significant risk is that the validation isn't consistent across the stack. For GapMap to give you reliable results, it needs to have correct information about tasks and schedules. If the validation rules aren't always followed between the frontend, backend, and database layers, bad or contradictory data could get into the system. For instance, tasks could be made without reasonable time limits, deadlines could be wrong, or timetable sessions could overlap in ways that weren't planned. These kinds of inconsistencies would make the recommendation engine less reliable and make it harder to fix problems. This means that validation is an important technical requirement, not just a small detail of the implementation.

An additional risk has to do with how much users trust the system. Users need to feel that the system's suggestions make sense and are easy to understand because

it is meant to give them suggestions rather than just store information. If the app gives suggestions that seem unclear, random, or impossible, users might quickly lose faith in the system and stop using it. This is very important for a project like GapMap, where the recommendation feature is meant to help people make decisions. A recommendation that isn't clear may make things more confusing instead of less. This is why transparency and explanation are important for both design and usability.

Even though there are risks, the project is still possible because its scope has been carefully limited. The minimum viable product does not include some high-risk stretch features like machine learning, integration with external calendars, and collaboration between multiple users. These exclusions make things a lot more possible. For instance, machine learning would make data collection, training, evaluation, and explainability more complicated, even though none of these things are needed to show the main value of the idea. Integrating external calendars would require using third-party APIs, authentication methods, and synchronisation problems, which would make development more difficult. If more than one person can work together, there will be more rules about managing users, permissions, and how to handle shared data. By not including these areas in the MVP, the project protects itself from having to deal with extra technical issues.

This planned narrowing of the scope makes the risks that have been found easier to handle. Limiting the feature set to the core task, timetable, and recommendation workflow is one way to deal with overscoping. Instead of using complicated predictive systems, you can improve the quality of recommendations by using clear rules-based logic and testing them over and over again. Regular checks at different levels of the application can help lower validation risks. Features that explain things and make sure that suggested tasks fit with the time window and task metadata can help build user trust. In each case, the risk is not ignored; instead, it is managed through practical design choices.



Risk	Why it matters	Mitigation
Overscope	Too many features reduce polish and reliability	Constrain MVP to core pages, CRUD, recommendation logic, and validation
Unrealistic recommendations	Users will stop trusting the product if gaps feel artificial	Use only real gaps between sessions and expose reasoning in UI
Validation inconsistency	Bad data reduces database integrity and recommendation quality	Validate on both client and server; keep rules documented centrally
UI clutter	A productivity tool that is slow to interpret defeats its own purpose	Use narrow page responsibilities, consistent cards, and restrained styling

PROTOTYPING AND ITERATIVE DESIGN

4.1 Low-fidelity concept

Before spending time on polished styling or detailed visual design, the low-fidelity prototyping stage is meant to figure out the system's basic information architecture, screen structure, and interaction priorities. At this point, the focus isn't on how the app looks, but on making sure it's logically organised, easy to use, and in line with the user's main goals. This is very important for GapMap because the system's value depends on how quickly users can go from entering information to getting a useful suggestion.

For GapMap, the low-fidelity prototype should have a simple and clear navigation structure that shows how the app's main features work. The system has a narrow focus on purpose, so the navigation should not be too complicated and should instead make it easy to get to the most important areas, like managing the timetable, managing tasks, getting an overview of the dashboard, and getting recommendations. At this point, a simple structure helps test whether the application flow makes sense and whether users can figure out where different actions go without needing visual polish to help them.

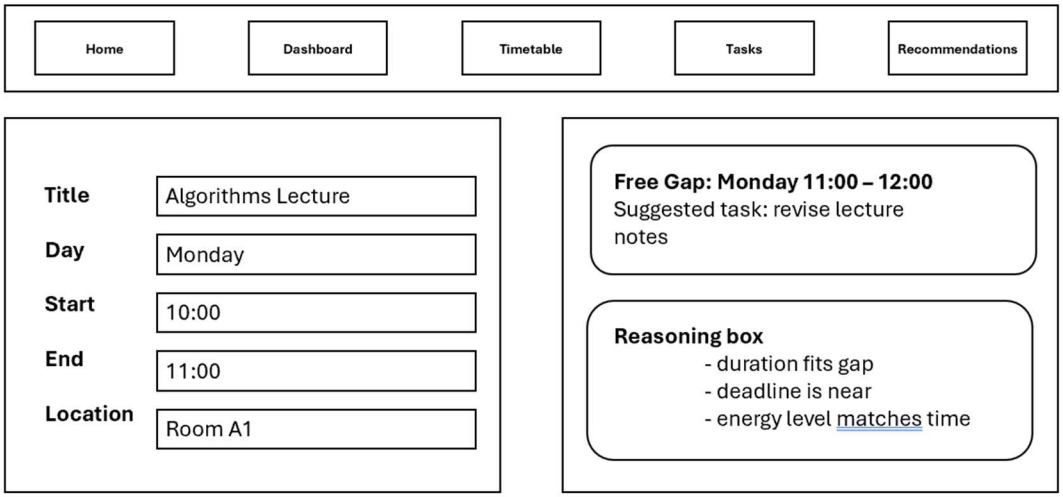
The timetable-entry form is another important part of the low-fidelity stage. The recommendation engine needs accurate timetable data to work, so the system needs to make it easy for users to enter sessions correctly and quickly. So, the prototype should look into how timetable inputs are arranged, what fields are needed, and how users go through the form. At this point, the goal is to find out if the form makes it easy to enter data and if the way it is set up makes sense for the types of sessions that users are likely to record, like lectures, seminars, work shifts, or other fixed commitments.

The recommendation area is another important area to pay attention to. This part is very important to GapMap's identity, so it needs to be clear from the start of the design process. The prototype should not only show where the recommendations will be, but also how the extra information will be shown next to them. It should explain not only what task is being suggested, but also why it was chosen. This is important because the reason behind a recommendation is part of how the system builds trust. People are more likely to trust and follow suggestions if they can see that they are based on clear things like how much time is available, how urgent the task is, or how much effort it will take to do it, rather than seeming random.

Low-fidelity prototyping is useful here because it lets you quickly explore and test these ideas without having to commit to a detailed implementation too soon. You can look over and change the design before you spend time on styling or technical details by drawing out the layouts, workflows, and content hierarchy in a simpler way. This lowers the chance of making an interface that looks good but is poorly organised. It also lets peers or stakeholders give feedback earlier, which may make it easier for them to comment on layout and interaction when visual details don't get in the way of the design decisions.

Figure G. Reconstructed low-fidelity wireframe

Paper-style concept showing a simple navigation bar, task form and recommendation area



4.2 Medium-fidelity prototype

The medium-fidelity prototyping stage builds on the structural work done during low-fidelity design by making the layout, hierarchy, and interaction emphasis clearer. The goal at this point is not to make a fully polished final interface, but to go beyond rough sketches and make a model of the system that better shows how the finished application will work in real life. This step is important for GapMap because it helps test if the interface makes its priorities clear and if users can follow the intended workflow with little trouble.

One of the main goals of the medium-fidelity prototype is to improve the visual hierarchy. This means making sure that the most important information and actions are easy to see on each page. Users of GapMap need to quickly figure out where to add timetable sessions, manage tasks, and see recommendations. The medium-

fidelity design should make these differences clearer by making better layout choices, stronger sectioning, and more careful placement of buttons, headings, and content areas. This helps make sure that the user focuses on the most important parts of the interface.

At this point, another area that needs work is spacing and grouping. Low-fidelity prototypes show what content is there and where it is roughly located. Medium-fidelity work, on the other hand, starts to shape how that content is organised in a way that makes sense and is comfortable. For instance, using cards and panels to group similar information can make the dashboard, task displays, and recommendation areas easier to understand. Better spacing makes things look less cluttered and makes it easier to scan quickly. This is especially important for GapMap because users are likely to use it in short bursts between other tasks. Before the final styling is done, a cleaner structure makes the system seem more professional and easier to use.

The navigation system also becomes clearer. Compared to the first sketches, the labels should be clearer, more specific, and easier to read. This is important because navigation should make it clear what each section is for right away, not make the user guess what it is for. At the medium-fidelity stage, the names and locations of pages like Dashboard, Timetable, Tasks, and Recommendations should be stable enough to see if the overall information architecture works. If users still have trouble figuring out where actions go, this stage gives you a chance to change the wording and layout before the implementation is done.

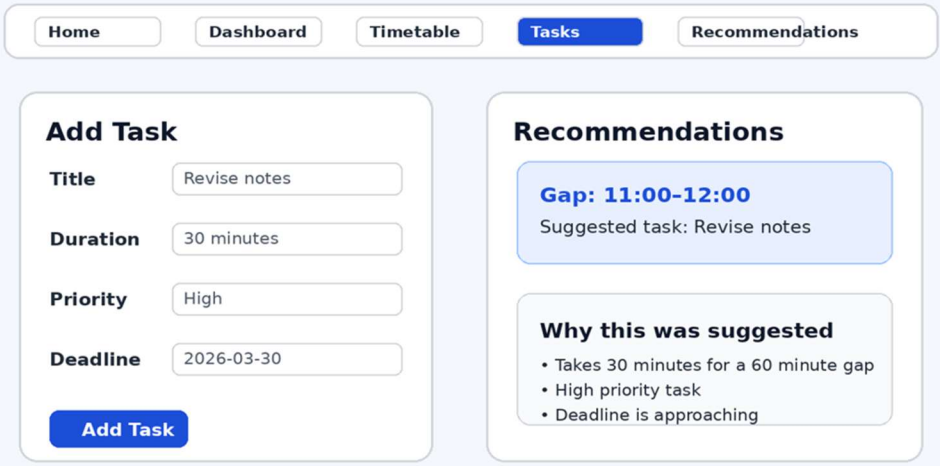
Another important area of focus is the recommendation area, which should now start to look like a real conversation instead of just a placeholder. The medium-fidelity prototype should not just say where a recommendation will be, but also show how it will be presented, what supporting information will be with it, and how the explanation will be given. This is especially important for GapMap because the app's main feature is its ability to make recommendations. The user should be able to see not only the task they chose, but also the reasons for it, like how it fits with the time they have or how urgent the deadline is. At the medium-fidelity stage, showing this more realistically helps test whether the recommendation output is clear and trustworthy.

At this point in the design process, the user should also be able to understand how the application works as a whole, even if the backend logic isn't fully in place yet. The prototype should be clear enough for someone to see how they would move through the system, such as entering timetable data, managing tasks, seeing dashboard summaries, and getting recommendations. The medium-fidelity prototype

is a useful link between the idea and the final product. It lets you test the interaction model in a way that is much more like the real product, but it still leaves room for changes before the final development decisions are made.

Figure H. Reconstructed medium-fidelity prototype

A cleaner component-based layout with cards, stronger hierarchy and colour-coded action



4.3 High-fidelity prototype

The high-fidelity prototype is the point at which the interface starts to look like the GapMap system's almost-finished visual language. The high-fidelity design is different from earlier prototype stages because it shows how the finished application is supposed to look and feel in real life. Earlier stages focused mostly on structure, flow, and interaction priorities. At this point, visual consistency, clarity, and a professional look are very important because the prototype should look a lot like the final product.

The high-fidelity interface for GapMap should have a simple, purposeful design style that is typical of productivity apps. To keep the text readable and stop it from looking stretched or hard to scan on bigger screens, the layout should use a centred page width. This makes the presentation more controlled and balanced, which helps the user focus on the most important information without getting distracted. Because the app is meant to be used quickly and often, the structure needs to be clear rather than visually complicated.

There should also be a strong focus on card-based design that is consistent. Cards are a helpful way to group and separate related information like task details, dashboard summaries, timetable sessions, and recommendation outputs. When the styling of cards is the same, the interface feels more consistent and predictable, which makes it easier to use. When similar kinds of information are shown in a consistent way, users can understand the interface faster and get used to the system over time. This is especially useful in GapMap, where the user might only be able to use the app for a short time between other tasks and needs to be able to recognise it right away.

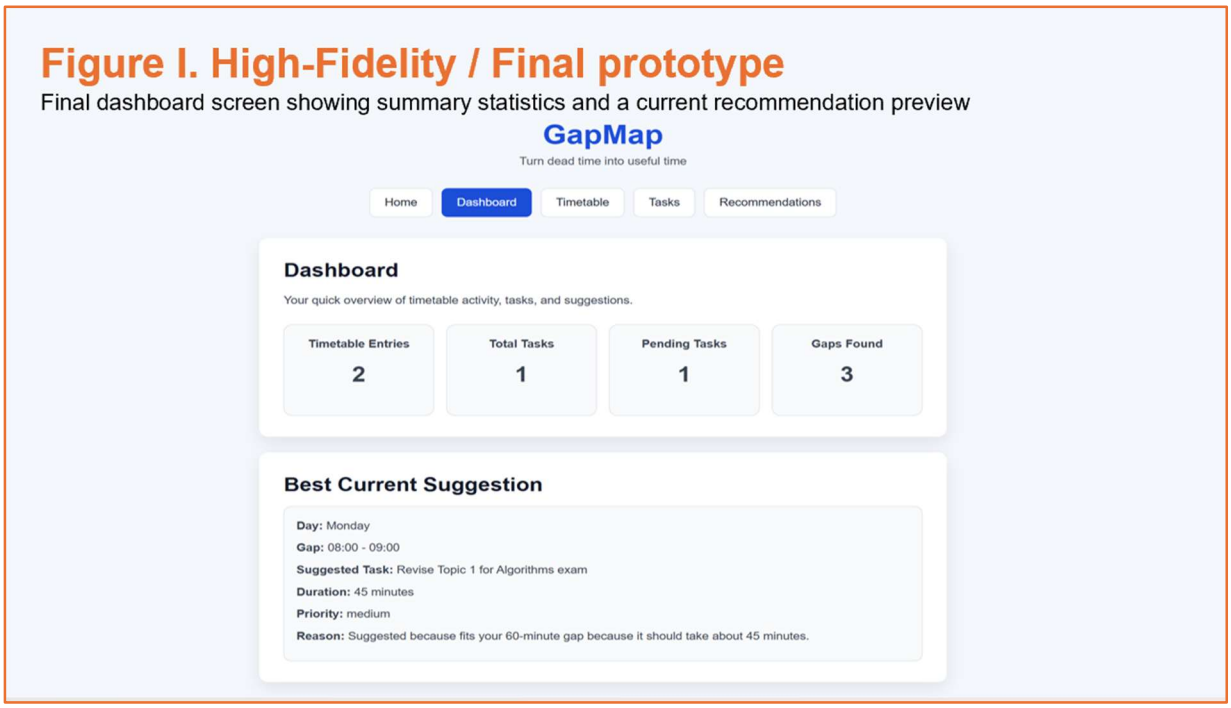
Additionally, there should also be clear and well-defined action buttons in the design. It should be easy to find buttons for adding tasks, making timetable sessions, changing the status of a task, or making suggestions. They should also look different from the other content around them. Their location and size should show how important they are on each screen. In an interface that focuses on productivity, actions should be clear and direct, not hidden or unclear. This helps people use the app more efficiently and cuts down on unnecessary mental effort, which is in line with the app's main goal.

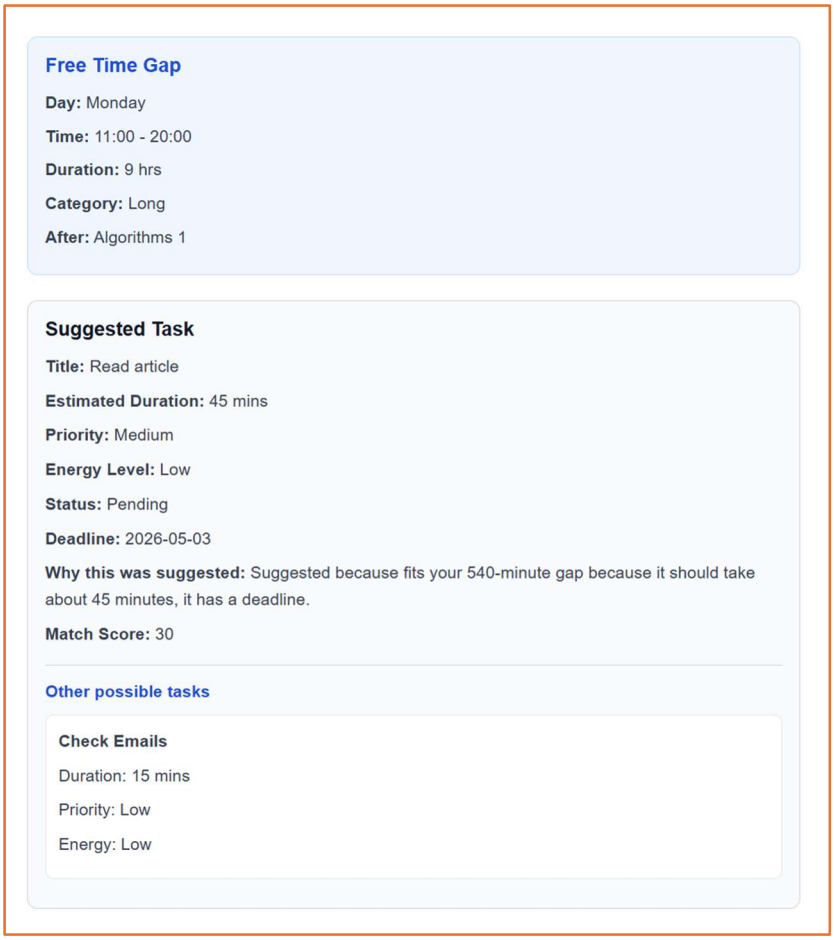
Using balanced whitespace is another important design rule at the high-fidelity stage. Adequate space between interface elements makes it easier to read, cuts down on visual noise, and makes the user experience calmer and more focused. In GapMap, whitespace isn't just for looks; it also helps users quickly scan information and tell the different parts of the interface apart. A design with good spacing can make the system feel more user-friendly and easier to use, especially when people are using it in short breaks during a busy day.

Since GapMap is a productivity app, the system's visual polish should come from clarity and restraint, not from newness for its own sake. You don't need to use very experimental layouts or fancy visual effects to make the interface look good. Instead, quality should be shown by paying close attention to spacing, typography, alignment, and visual hierarchy. These things have a much bigger effect on how easy it is to use than extra decorations. Clear typeface makes it easier for people to read tasks, deadlines, and suggestions. A strong hierarchy makes sure that the most important information stands out. Consistent spacing and alignment give the app a sense of order that helps it do what it's supposed to do: make things more productive.

Because of this, the high-fidelity prototype should show not only what the final system will look like, but also what its design values are. It should show that GapMap is meant to be calm, focused, and useful, not too busy. This is very important because users need to be able to quickly trust and understand the recommendation

feature. A clear and organised interface design builds on that trust by making the system seem reliable, well-organised, and carefully planned.





4.4 Design heuristics and accessibility

To make the GapMap interface easier to use, clearer, and more trustworthy, it should be based on established principles of human-computer interaction. Using well-known design heuristics helps make sure that the system works and is easy to use. This is very important for GapMap because the app is meant to help people make decisions quickly. The recommendation feature would be much less useful if the interface is hard to understand, inconsistent, or confusing.

Nielsen's usability heuristics are an important set of rules that help with design. These principles stress the need for the system's status to be easy to see, the interface to behave consistently, the system to use the user's own language, the system to stop users from making mistakes, and the system to help users recognise things instead of having to remember them. When it comes to GapMap, visibility of system status means that users should always know what the app is doing, whether an action worked, and whether any problems need to be fixed. For instance, when a task is added, a timetable session is deleted, or a validation rule is triggered, the system should give feedback right away that is easy to understand. This makes things less confusing and helps the user feel in charge of the conversation.

Consistency is just as important. Users shouldn't have to relearn patterns as they move between pages, so labels, layouts, and actions should all work the same way throughout the app. Instead of using vague or overly creative language, use common words like "Tasks," "Timetable," and "Recommendations." This will help keep the system and the language users naturally expect in sync. Recognition rather than recall is also very important. Users shouldn't have to remember hidden commands, old states, or complicated workflows. Instead, the interface should make actions and options clear, easy to find, and easy to understand.

Norman's design principles, which stress the importance of feedback, limits, and actions that can be found, support these ideas. Feedback makes sure that users know what will happen if they do something wrong, and limits help keep invalid input or wrong interactions from happening. Discoverable actions are very important in GapMap because the system should be easy to use so that people can quickly find their way around it, even when they are busy with other things. Buttons should look like they can be clicked, important actions should stand out from the rest, and helpful text should be added when needed to clear up any confusion. For instance, light supporting text might be helpful on pages where the user needs help figuring out what information to enter or how recommendations are made.

These principles give GapMap some specific design requirements in real life. There should be clear success and error messages in the system so that users can tell if they did things like saving, deleting, or updating data correctly. Labels for navigation should stay normal and easy to understand right away. Buttons and form controls should have clear affordances so that users can easily tell which parts of the page are interactive. Pages that have more advanced features, like generating recommendations, should also have short explanations so that users can understand what the page is for and how the results were reached.

Accessibility should be thought about from the start of the design process, not added later as a change. Making things accessible from the start makes them easier for everyone to use and lowers the chance of making barriers that are hard to fix later. For GapMap, this means that when making decisions about the interface, you should think about how easy it is to read, how clear it is, and how easy it is to use in a variety of situations.

There are a few specific things to think about when it comes to accessibility. The app should use the right colour contrast so that text and other interface elements are easy to read. Users should be able to easily see what information is needed in form fields. Buttons and other clickable parts should be big enough to use comfortably, especially on tablets and other touch-capable devices. The font sizes should be easy to read without too much zooming in, and colour should only be used when absolutely necessary. The system shouldn't rely on colour alone to convey meaning, as this could leave out users who have trouble seeing colours or who are using the app in less-than-ideal conditions.

The project's non-functional requirements should also clearly show that these things are important. The app should work on most laptop and tablet screen sizes, and the layouts should change to fit without losing clarity or functionality. This is especially important because people will probably use GapMap on different devices and in different situations, and usability shouldn't depend on just one screen format. The project shows a more responsible and professional way to develop interfaces by making accessibility a part of the requirements and design rationale.

4.5 Planned iterative refinements

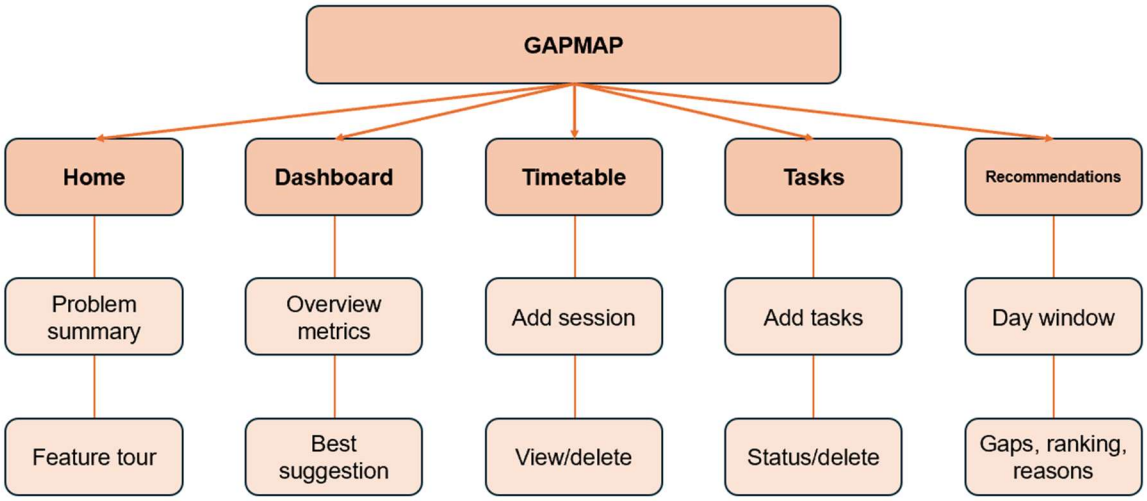
Observed weakness or design risk	Planned design response	Expected improvement
Recommendation output may feel arbitrary	Show a reason string and alternative suggestions	Improves trust and transparency
Users may perceive the app as another cluttered planner	Keep to five pages with one dominant purpose each	Reduces cognitive load
Short-gap tasks may still be hard to judge visually	Use card format to show duration, priority and deadline clearly	Improves scalability at decision time
Repeated re-entry of day-window preferences may frustrate users	Persist active day window locally	Improves convenience and continuity

FORMAL SYSTEM DESIGN

5.1 Information architecture

Figure J. Information architecture / site map

Top level pages and their main responsibilities in the proposed MVP



GapMap's information architecture is meant to be shallow, clear, and focused on tasks. One of the main goals of the project is to make planning easier, not harder. This decision is in line with that goal. The application is designed to help users make quick, useful decisions when they only have a little bit of time, so the system needs to be set up so that users can go straight to the function they need without having to go through a complicated or dense interface first. A shallow architecture helps the project as a whole by making it easier on the brain.

Figure J shows that the proposed minimum viable product is made up of five main pages: Home, Dashboard, Timetable, Tasks, and Recommendations. Each of these pages has a clear job to do, so the user can quickly understand what each part is for. This separation of concerns is important because it keeps the interface from getting too busy with too many unrelated functions in one place. Instead of making the user figure out what to do with a big, all-purpose dashboard, the system gives each main feature its own space.

The Home page is the first page you see when you open the system. The goal is to explain the problem that GapMap is meant to solve and give a brief overview or tour of its features. This is especially helpful for people who are using the app for the first

time because it helps them understand what the app is for before they start entering data. The Home page is not for managing tasks that are happening right now; instead, it helps with orientation and understanding.

The Dashboard page shows a summary of the user's current situation. The site map shows that this includes summary metrics and the best suggestion at the moment. The dashboard's job is not to duplicate the full functionality of other sections, but to give a quick overview of decision support. It should let the user see their workload and the most relevant recommendation right away, without having to dig deep into the app. In this way, it serves as a central snapshot instead of a place to do detailed management.

The Timetable page is in charge of keeping track of scheduled sessions. Its main jobs are to add new sessions and look at or delete old ones. This page helps the system's schedule-awareness by making it easy to record fixed commitments in a clear and organised way. It is important to accurately identify real gaps between sessions in order to give good recommendations. Putting timetable management in its own area helps keep things clear and supports reliable user input.

The Tasks page does the same thing for work items. It lets the user add tasks and keep track of their status or delete them. It's helpful to keep task management separate from timetable management because they are two different types of user input. One is about fixed time commitments, and the other is about flexible work that needs to be done. The app makes things less confusing by separating these areas, which helps users better understand how the system works.

The Recommendations page is the most unique part of the app. The proposed structure says that this page should have the active day window and the recommendation output, which should include gaps that were found, a ranking, and reasons for the ranking. Putting recommendations on their own page is an important design choice because it makes the feature the main value of the system instead of just a small add-on. It also gives you enough room to show not only the chosen task but also the reasons for it, which is important for trust and openness.

This overall structure is a deliberate design choice. Each major feature is given enough independence to stay clear, and the total number of sections is small enough to make navigation easy. So, the architecture isn't too deep or too broken up. It can be used by people who are new to it and by people who use it every day, which is important for a system that needs to be easy to get to quickly between other tasks.

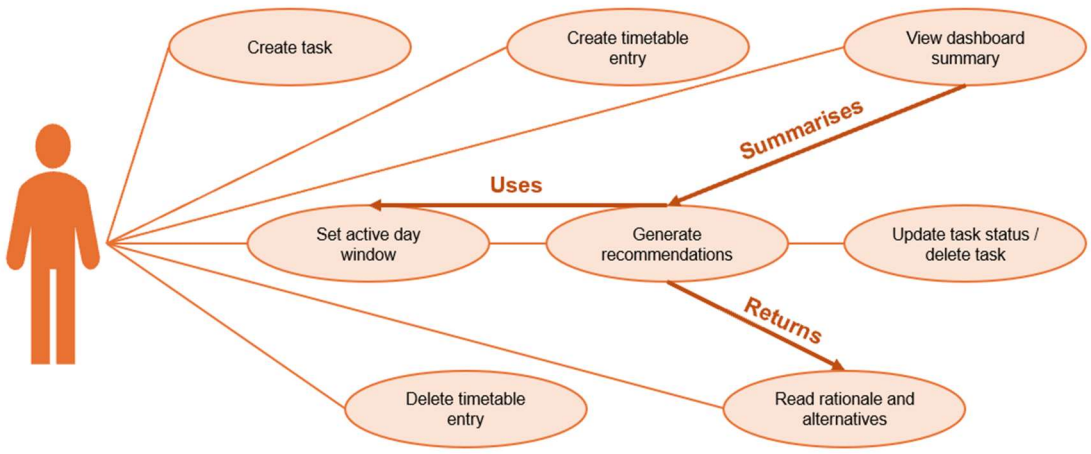
From the point of view of human-computer interaction, this shallow structure makes recognition easier than recall by making the system's main functions clear and easy

to guess. The top-level navigation shows users the main actions of the app, so they don't have to remember where complicated nested options are. This is especially true for GapMap, where the system's value comes from making it easy for users to get things done quickly.

5.2 Use case specification

Figure K. Use case diagram

Primary user interactions expected from the MVP design



The GapMap minimum viable product's use case model is purposefully focused on one main actor: the student user. This is a deliberate design choice meant to keep the system focused on its main goal while avoiding adding extra complexity at this stage of development. Adding more roles, like administrators, analysts, or coordinators, would make modelling more difficult without helping with the main design problem. Because the system is meant to be a personal productivity tool and not a multi-role enterprise platform, focusing on one actor makes the specification clearer and more useful.

This one-actor approach is also in line with the MVP's goals. The goal of GapMap is to help each user keep track of their own schedule, tasks, and suggestions. So, the most important interactions are all things that the student user does directly to keep their personal information up to date and get useful help with decisions. By focusing

the use case model on this one role, the design stays focused on the real user journey instead of getting lost in ideas for features that might not happen.

The most important use case is adding tasks. This lets the user keep track of tasks that need to be done, along with the structured metadata that the recommendation system needs. The system wouldn't be able to suggest what the user should do during a free gap without this feature. Another example of this is the use case for entering timetable sessions, which lets the user write down things like lectures, seminars, work shifts, or other planned events that they have to go to. This is just as important because the recommendation engine needs timetable data to find real free times between commitments.

Another important use case is setting the active day range. This lets the user set the hours that should be considered part of their normal workday. Adding this use case makes recommendations more useful because it stops the system from treating every unscheduled time as equally valid for work. Instead, suggestions can be limited to the time frame that the user thinks is available for productive work. This makes the system more tailored to each user and less likely to give you results that aren't useful.

The main purpose of the app is to make suggestions. During this interaction, the system looks at the user's schedule, finds real gaps between sessions, compares those gaps to tasks that still need to be done, and makes the best suggestion. This use case is the main reason GapMap exists and the main reason for the other features. Managing tasks and schedules is important, but they mainly help the recommendation process.

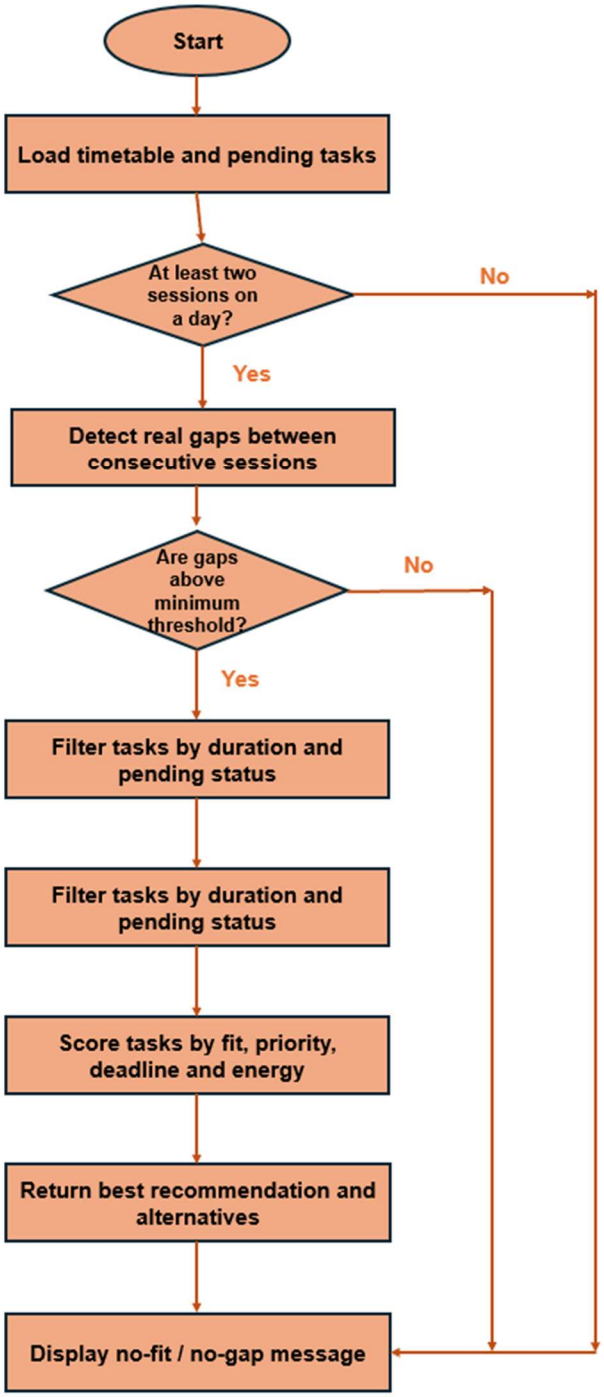
The use case of reading the reasoning behind the recommendation is just as vital. A recommendation in GapMap should not look like a black box output. Instead, the system should explain why a certain task was chosen, such as by showing that it fits the time frame, meets the deadline, or matches the expected level of effort. This use case is important because it shows how the system builds trust. If the reasoning behind a recommendation is clear and easy to understand, the user is more likely to accept and trust it.

When you put all of these use cases together, they show a focused and consistent interaction model. The student user gives information about tasks and schedules, sets the active day window to show the relevant working context, asks for or looks at recommendations, and figures out how the system came to its conclusion in order to decide. This order shows the app's main value without adding any extra roles or interactions that aren't part of the MVP.

5.3 Activity design

Figure L. Activity flowchart for recommendation generation

Only real gaps between sessions are used; unsuitable tasks are filtered before ranking



The activity design for GapMap makes the logic behind how recommendations are made official and, more importantly, sets the rules for when a recommendation

shouldn't be made. This is an important design choice because the system's value depends not only on making suggestions, but also on making sure those suggestions are realistic, believable, and appropriate for the situation. If recommendations were made too loosely, the app could easily suggest work at times that don't make sense, which would make users less likely to trust it and make it less useful.

As illustrated in Figure L, the recommendation process begins by loading the user’s timetable data together with their pending tasks. After this, the system runs a series of checks and filters before any ranking happens. The first big restriction is that the system should only make a recommendation if it can find at least two timetable sessions on the same day. This rule is important because the project only makes recommendations about real gaps between sessions, not just any time that isn't scheduled. The system could think that the start of the day before the first session or the end of the day after the last session is a valid recommendation window if this condition is not met. These long open periods may not cost anything in terms of technology, but they don't fit the definition of the "dead time" that GapMap is meant to help with.

The system can only find real gaps between consecutive entries in the timetable once it has confirmed that there are at least two sessions. After that, these gaps are tested against a minimum threshold to get rid of very short intervals that probably won't help with completing tasks. This step stops the app from giving suggestions for gaps that are too short to be useful. The system shouldn't try to make a recommendation if it can't find a valid gap. Instead, it should send a clear message saying "no gap" or "no fit," which would make the result clear instead of seeming broken or random.

If there is a good gap, the next step is to sort the tasks by whether they are still open and whether the time they are expected to take fits with the time you have. This filtering step is very important because it gets rid of options that are clearly not good before ranking starts. The system makes both the efficiency and the quality of the recommendations better by first narrowing the candidate set. Only tasks that can realistically fit into the time you have left and that you haven't finished yet should be suggested.

After filtering, the tasks that are left are scored based on things like how well they fit, how important they are, how urgent the deadline is, and how much energy they need. This scoring stage shows what GapMap is really all about: helping people make decisions. The system is not just finding any task that is open; it is also trying to figure out which one makes the most sense given the gap that was found. The

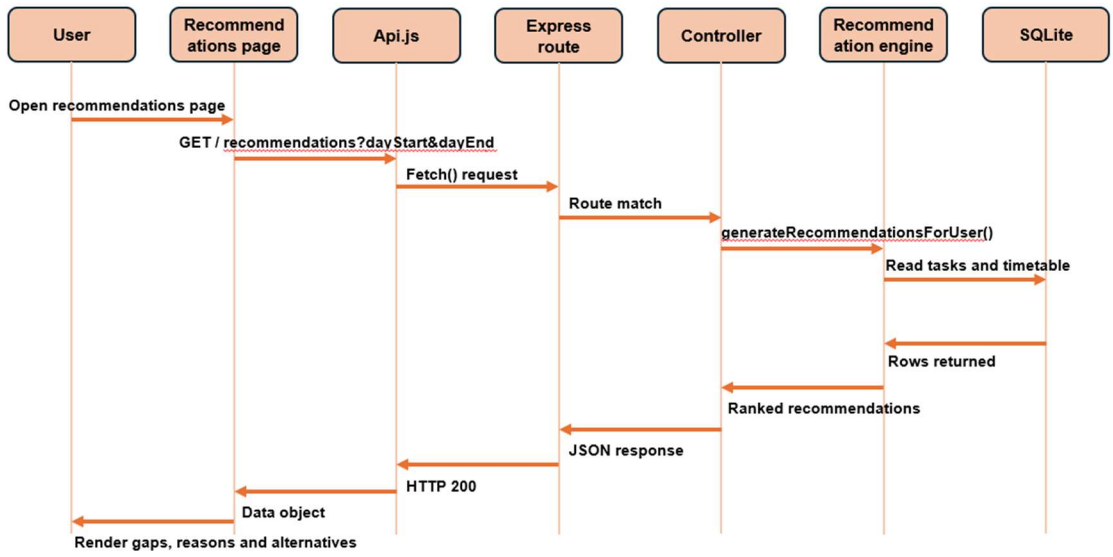
recommendation logic can make a better output by combining these factors than just using the first match. The top-ranked task is then given as the main suggestion, along with one or more other options if they are needed.

One important part of this activity design is that it includes failure conditions in a planned and controlled way. The logic doesn't always make a recommendation; instead, it knows when it shouldn't. This makes the system stronger and more reliable. A good productivity app should be able to explain when it can't make a good suggestion instead of giving a weak or misleading answer just to avoid an empty state.

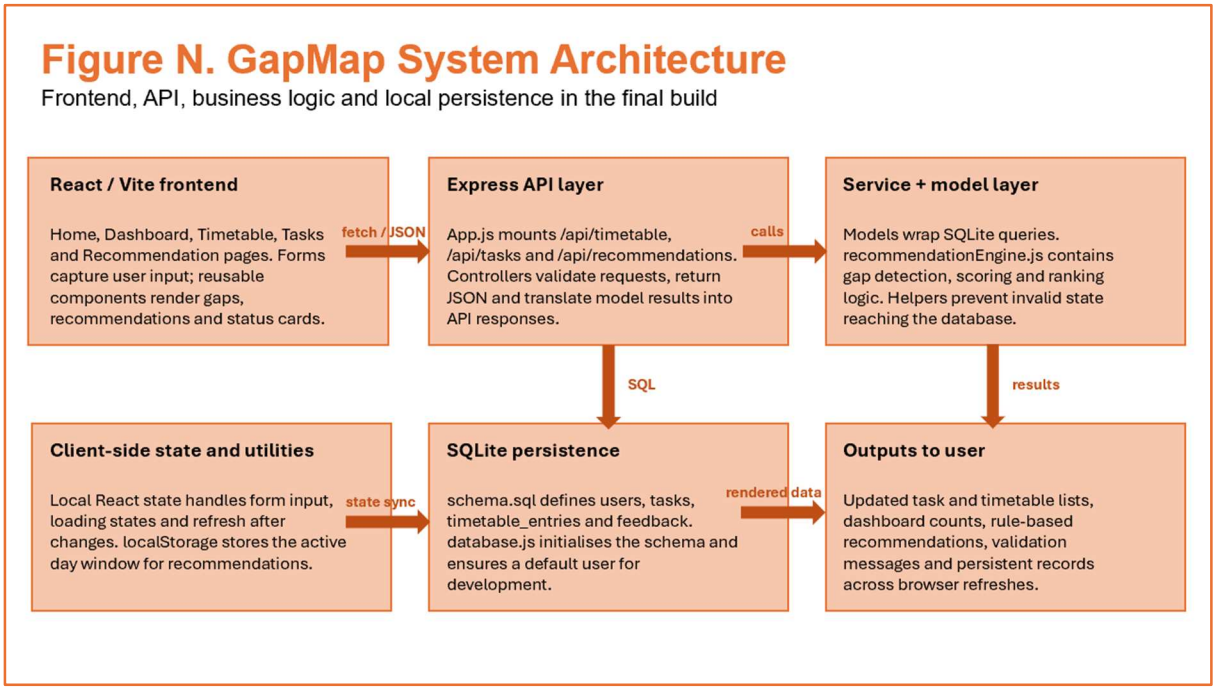
5.4 Sequence design

Figure M. Sequence diagram for fetching recommendations

A single request travels through the React client, Express API and recommendation service



5.5 Technical architecture



GapMap's technical architecture is based on the standard client-server model, with a clear division of responsibilities between presentation, application logic, and data storage. This architectural choice is good for the project because it makes it easier to maintain, allows for modular development, and clearly divides up responsibilities without adding extra complexity. Figure N shows that the system is made up of different layers that work together to make the whole application workflow happen, from the front end where users enter data to the back end where recommendations are made and data is stored permanently.

The React/Vite frontend is in charge of displaying the user interface and handling client-side interaction at the presentation layer. This includes moving between the system's main pages, getting input from forms, managing local state, and showing tasks, timetable entries, dashboard summaries, and recommendation outputs. React is a good choice for this job because it supports reusable components and reactive updates. These are both useful in a productivity app where users may often add, edit, or delete data and expect the interface to update right away. Vite is a lightweight and efficient frontend development environment that works well for a project of this size.

An Express API layer connects the frontend to the backend. This layer connects the user interface to the data and service logic that it is built on. It is in charge of getting requests from the client, checking incoming data, sending requests to the right

controllers, and sending back structured JSON responses. This separation is important because it keeps the frontend from being too closely linked to business rules or data access logic. Instead, the frontend is still in charge of how things look and how people interact with them, while the backend is in charge of processing and checking. This makes the system more modular and professionally organised.

The service and model layer is behind the API layer. This is where the system's main business logic is put into action. This is the part of the architecture that makes GapMap technically unique. This is where the recommendation engine is, and it does things like finding real gaps between timetable sessions, filtering out tasks that aren't appropriate, scoring the remaining candidates, and ranking them based on rules that have been set. The model layer also wraps access to the database and makes sure that interactions with data that stays the same are still organised and controlled. You can use helper functions to enforce rules and keep bad application state from getting to the database. So, this layer is important for both the system's functionality and its reliability.

SQLite is used to store the main entities that the application needs at the persistence layer. These are things like tasks, schedule entries, and any other records that the system needs to work. SQLite is a good choice for the GapMap MVP because it lets you store data locally without the extra work that comes with a more complicated database server. SQLite has enough features to support structured storage, querying, and retrieval for a single-user academic project that is developed and run locally. It is also easy to set up and use with other programs. This helps the project reach its goal of making a technically sound system without going overboard with the persistence layer.

Figure N also shows how client-side state and utilities help improve the user experience. Local React state takes care of things like loading states, form input, and how the interface refreshes after updates. You can use more local storage for things like the active day window to make sure that some of your preferences stay the same between browser sessions. This makes the application easier to use by cutting down on the need to set it up again and making it easier to get back to where you were when you reopen it.

The overall architecture is good because it meets its needs without adding unnecessary complexity. To reach its goals, GapMap doesn't need a distributed microservice architecture, real-time messaging infrastructure, or an enterprise-scale deployment model. A modular client-server design, on the other hand, strikes the right balance between ease of use and technical quality. Each layer has its own job, dependencies are easier to understand, and development can move forward in a

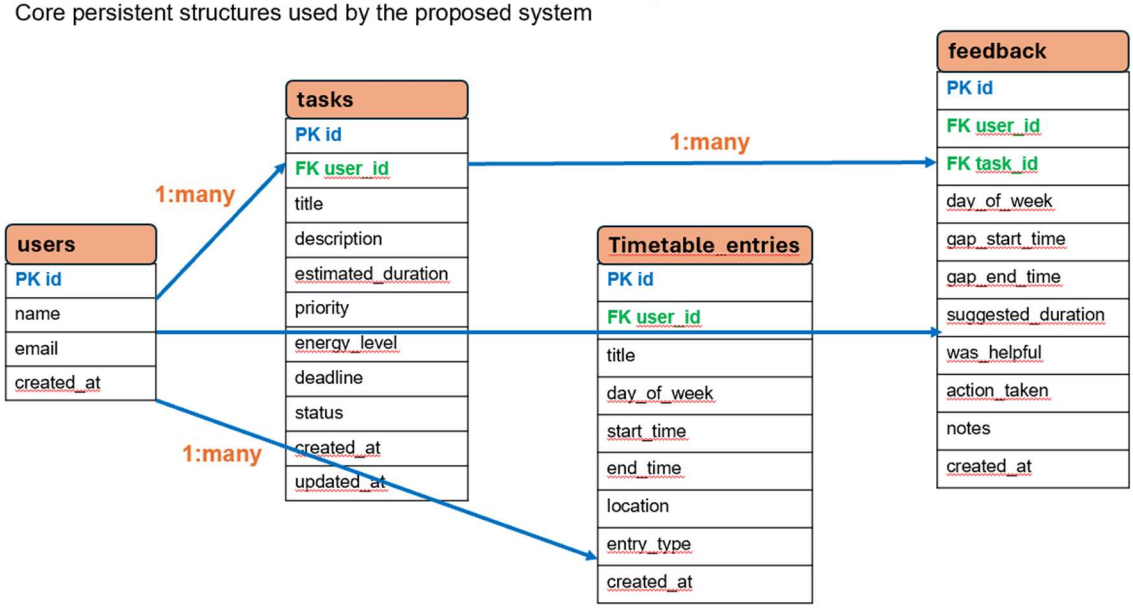
structured way, step by step. This also makes it easier to test and fix problems because it's easier to find the part of the system that caused the problem.

This structure also shows good modularity from a software engineering point of view, which helps the project's technical quality. The project brief says that modularity is an important part of good system design. The GapMap architecture supports this by clearly separating interface code, API handling, business logic, and database operations into different areas. This not only makes the implementation better right away, but it also makes it more likely that it will be extended in the future. Because the foundation is organised in a disciplined way, it will be easier to add features like authentication, external integrations, or better recommendation strategies later on without causing too much trouble.

5.6 Data design

Figure O. Entity relationship diagram

Core persistent structures used by the proposed system



The data design for GapMap is small on purpose, but it can do a lot. This is in line with the project's overall goal of creating a focused and technically sound minimum viable product instead of a system that is too big and complicated. The database structure only has a few main entities, but each table has a specific job to do to support the application's main features, such as analysing timetables, evaluating tasks, making recommendations, and later evaluating the system. Figure O shows that the proposed entity relationship model gives persistent storage a clear and

manageable structure while still keeping track of the information needed to support the system's logic.

The users table is at the heart of the model. It gives the system a stable identity anchor. Even though the MVP currently uses a default local user instead of a full authentication model, including a user entity is still an important design choice. It sets up a good base for data ownership, lets related records be linked in a consistent way, and makes it more likely that future extensions will be possible if authentication or multi-user support are added later. Even when there is only one user, a defined user record makes the relational structure cleaner than putting all the data together without ownership context.

The tasks table stores the information required to represent work items that may later be recommended by the system. There are fields in this table for the title, description, estimated duration, priority, energy level, deadline, and status. These aren't random additions; they help make the recommendation process better. Estimated duration helps figure out if a task can realistically fit into a gap that is open. Priority and deadline give you a way to decide how important and urgent something is. Energy level lets the system show effort or intensity, which can help make suggestions that are more relevant to the situation. Status is also important because the recommendation engine should only work on tasks that are still open and not ones that have already been finished. This is how the task model is closely linked to the system's main decision-support function.

The timetable_entries table gives you the time frame you need to find gaps. This table holds organised information about fixed commitments, such as the day of the week, start and end times, title, location, and type of entry. Instead of making vague guesses about free time, these records let the system find real gaps between sessions. The timetable data is what makes GapMap unique. It lets the app know when the user is actually busy and when there might be good opportunities to work. Without this table, the recommendation feature wouldn't be able to keep track of schedules.

The feedback table goes beyond the model's current features and helps with evaluation and future improvements. It keeps track of things like the user involved, the task at hand, the day and time of the gap, the suggested duration, whether the suggestion was helpful, what action was taken, and any notes the user left. Adding feedback to the design is helpful because it gives you a way to see how useful the recommendations are over time. Even though this feature is limited in the MVP, it shows that the team is thinking ahead and will help them think about how well the system works in real life later on. It also makes the project more likely to be

evaluated well because it allows for structured evidence collection instead of just relying on informal impressions.

The links between these tables are meant to keep things consistent and make sure that processing is reliable. The ERD shows that one user can have many tasks, many timetable entries, and many feedback records. There is also a link between tasks and feedback, which lets the system keep track of responses that are related to certain suggested tasks. This relational structure helps make sure that records are linked in a meaningful way instead of just being separate data points.

Referential integrity is a very important part of this design. The system shouldn't work with records that are malformed, inconsistent, or orphaned because GapMap's recommendation logic depends on valid data relationships. For instance, there shouldn't be a feedback entry without a valid user and task, and a task shouldn't be separate from the user it belongs to. If these rules aren't followed, the recommendation engine and evaluation processes might give results that aren't accurate or helpful. Referential integrity is important not just for databases, but also for the quality of applications. It helps keep the system as a whole logically sound.

In this project, the schema's small size is a plus. A compact model is easier to use, check, and test, but it is still expressive enough to support the whole recommendation workflow. The design also shows the project's broader commitment to scope discipline by only including the data structures needed for the main problem and avoiding unnecessary ones. This helps keep the architecture clean and lowers the chance of overengineering.

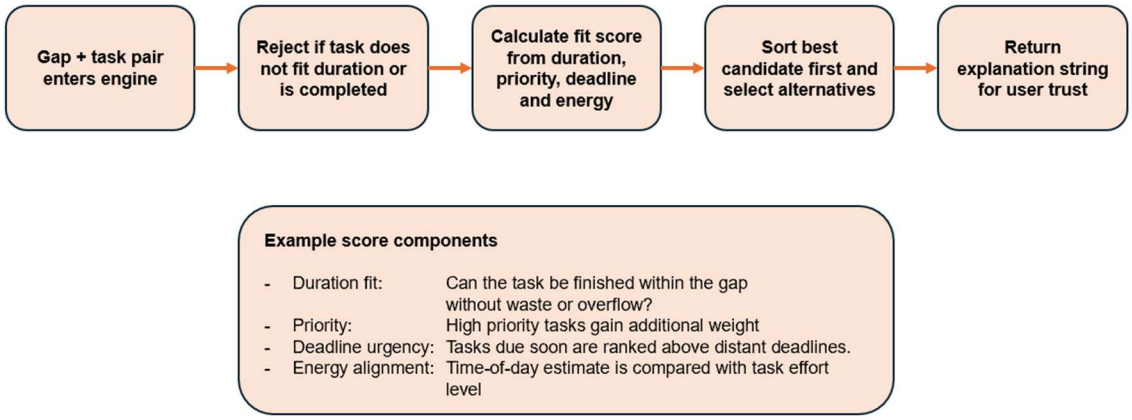
5.7 API design overview

Route	Method	Purpose	Primary response
/api/tasks	GET	Retrieve all tasks for the active user	List of task objects
/api/tasks	POST	Create a new task after validation	New task object or validation error
/api/tasks/:id/status	PATCH	Toggle or update task status	Updated task object
/api/tasks/:id	DELETE	Delete a task	Success message
/api/timetable	GET	Retrieve timetable entries	Ordered list of sessions
/api/timetable	POST	Create a new session	New timetable entry object
/api/timetable/:id	DELETE	Delete a timetable entry	Success message
/api/recommendations	GET	Generate ranked recommendations for current data and day window	Summary + recommendation objects

5.8 Recommendation engine design

Figure P. Recommendation scoring logic

Rules-based recommendation engine logic



The best design choice for the GapMap minimum viable product is a rules-based recommendation engine. This method works well for the project's size, goals, and limitations because it offers a recommendation process that is clear, can be tested, and can be done in the time frame that is available for development. A rules-based method, on the other hand, lets you clearly define, justify, and evaluate the system's logic. This is not possible with more complicated predictive or machine-learning-based methods. This is especially useful for an academic computing project, where clear design and the ability to follow the decisions made are just as important as the final product.

Figure P shows that the recommendation process starts when a valid gap and task pair enters the engine. The system is not trying to blindly evaluate all tasks at this point. Instead, it first filters out candidates who are clearly not a good fit. If tasks are already done or if their estimated duration doesn't fit in the available time slot, they should not be included. This first filtering step is important because it keeps unrealistic options from getting to the scoring stage and makes sure that only candidates who are truly qualified are looked at further.

Once unsuitable tasks have been removed, the system can evaluate the remaining candidates using a structured scoring process. There should only be a few clear factors that go into the score that show why the project was designed the way it was. One of the most important is duration fit, which looks at whether the task can be done in the time available without going over or wasting too much time. Another

thing to think about is priority, which gives more weight to tasks that are marked as more important. A third factor is how urgent the deadline is. Tasks with deadlines that are coming up soon should be ranked higher than tasks with deadlines that are farther away. Energy alignment is the fourth factor. It compares the expected effort level of a task with the time of day or the expected state of the user. This helps the system avoid suggesting work that is too hard for the situation.

This method gives the recommendation engine complex decision logic without making the project make false claims about AI. That difference is important. GapMap is not designed to replicate intelligent forecasting using extensive behavioural data, nor does it necessitate such a degree of complexity to fulfil its purpose. The MVP's goal is to give clear and reasonable support for a well-defined use case. A well-designed rules-based system can fully meet this goal while being easier to put into action, debug, and evaluate rigorously.

Another major advantage of the rules-based method is that it supports the generation of a readable explanation string alongside the recommendation. The engine can give you not only the best candidate and relevant alternatives, but also a short explanation of why the top task was chosen after scoring and sorting. For instance, the system might say that the task you picked fits the available gap, is due soon, and is at the right level of effort for that time. This explanatory output is important because it helps users trust the system. A recommendation system that can't explain why it made a choice may seem random, even if its logic is sound. In contrast, people are more likely to trust, understand, and find useful a system that shows how it thinks.

The ranking stage should then sort the other candidates by score, with the highest-ranked task being the main recommendation and one or more alternatives being optional. This lets the system give clear directions while still giving users some freedom. So, the user is helped but not forced to choose one unclear outcome. This balance works for GapMap because the goal of the system is to help people make decisions without taking away their freedom.

The rules-based engine also has a lot of practical advantages from a software engineering point of view. The logic can be unit tested more easily and changed little by little if feedback from the prototype shows that the recommendations are too weak, too strict, or not intuitive enough. This is because each scoring factor is clear. This means that the engine can be improved over time as development goes on. The project's agile-style methodology works well with the fact that small changes to scoring weights or rejection conditions can be tested without having to redesign the whole system.

5.9 Validation and state integrity rules

Rule area	Rule	Rationale
Tasks	Title and estimated duration are required	Prevents incomplete work items entering the system
Tasks	Duration must be within a sensible range	Prevents impossible or meaningless recommendations
Tasks	Deadline cannot be earlier than today	Protects data quality and user trust
Timetable	Start time must be earlier than end time	Prevents impossible sessions and broken gap logic
Timetable	Day value must be valid and controlled	Protects ordering and grouping by day
Recommendations	Only real gaps between sessions are considered	Prevents unrealistic system output

TECHNICAL SPECIFICATION

6.1 Front-end stack

React with Vite is the suggested frontend stack for GapMap. This combination is perfect for the project's needs because it allows for modular interface development, fast local iteration during implementation, and efficient state-driven rendering. GapMap is meant to be a productivity app that runs in a web browser and has many pages that are linked to each other and interface elements that can be used again. A modern component-based frontend framework strikes the right balance between flexibility, maintainability, and development speed.

React is a good choice because it lets you break up the user interface into reusable and focused parts. The structure of GapMap works well with this. You can make dedicated page-level components for core pages like Tasks, Timetable, Dashboard, and Recommendations. You can also reuse shared interface elements like buttons, alerts, cards, and form elements throughout the app. This makes the system more consistent and cuts down on duplicate code. It also helps keep the overall design cleaner, which is important for a project where clarity and ease of use are key goals.

Another great thing about React is how it handles state. GapMap needs the front end to handle user input, loading states, validation feedback, and updates that happen when data changes. For instance, when a user adds a task, deletes a session from their schedule, or asks for a new recommendation, the interface should respond right away and show the new state of the app. Because React's reactive model automatically re-renders components when the underlying state changes, this behaviour is easier to implement and keep up with. This is very useful in a productivity app, where users expect to see right away that their actions have been completed successfully.

Vite works well with React because it gives developers a fast and light development environment. It is especially useful during iterative development, when interfaces may need to be changed often based on testing and feedback, because it starts up quickly and has a good hot-reload behaviour. This speed of local iteration is useful because GapMap is being developed as part of an academic project with a short time frame. It lets you quickly test and improve interface ideas without making the tools more complicated than they need to be.

This frontend stack is also a good choice because it fits the size of the project. You don't need a complicated frontend architecture or a heavy framework to use GapMap. React with Vite can handle all the features needed for the MVP while still being easy to use, well-documented, and accessible. This is a smart technical choice that helps both the development process and the project's long-term readability.

6.2 Back-end stack

Node.js with Express is the suggested backend stack for GapMap. This is a good choice for the project because it offers a lightweight and well-established way to build a JSON-based API that is still simple enough to implement, test, and explain clearly within the module's scope. A Node and Express backend is a good choice for GapMap because it needs structured communication between a browser-based frontend and a persistence layer. This helps with both development speed and architectural clarity.

Node.js is a good choice because it lets the project use JavaScript on both the front end and the back end. This keeps development consistent and cuts down on the extra work that comes with switching between programming languages. This can help you get more done while you're implementing it and make the whole system easier to understand. For an academic full-stack web project like GapMap, this consistency is very helpful because it makes debugging easier, speeds up the process of making changes, and keeps the codebase more organised.

Express gives you a simple and clear way to handle HTTP requests and responses. This is important for GapMap because the backend is mostly an API layer that connects the user interface to the data and recommendation logic that powers it. For things like making tasks, deleting timetable entries, changing the status of a task, and getting recommendation results, the app needs to be able to handle structured requests and responses. Express is a good choice for these needs because it lets you easily define routes, use middleware, and organise your code around controllers without adding unnecessary abstraction.

Another good thing about this backend choice is that Express routes fit well with the domain objects that are already in the system design. You can easily organise routes for tasks, timetable entries, and recommendations around the application's main resources. This makes the API easier to use and keep up with because each endpoint represents a useful part of the system instead of an arbitrary technical

structure. A resource-oriented route design also makes it easier to document and test the backend and makes it easier to tell who is responsible for what.

In this structure, controllers are the main places for validation and orchestration. Their job is to get requests, make sure the data is correct, coordinate the necessary actions, and send the right responses back to the client. This is an important design choice because it keeps routing definitions from getting too full of processing logic. Instead, routes stay pretty clean, and controllers take care of the actual flow of each request. This makes it easier to read and supports a more professional backend structure.

A dedicated service layer is especially useful for GapMap because it keeps the logic for making recommendations separate from the routing of API requests. The recommendation engine is the most unique and technically interesting part of the app. Because of this, it should be in a layer where it can be developed, tested, and improved without affecting how HTTP requests are handled. The project is more modular and easier to keep up with when business logic is separated from routes and controllers. This also makes testing easier because you can check recommendation rules without having to rely on full API interactions.

6.3 Database stack

The proposed database stack for GapMap is SQLite, which was chosen as the minimum viable product's persistence layer. SQLite is a good choice for this because it is lightweight, reliable, and more than enough for a local single-user app of this size. The project doesn't need the infrastructure or operational complexity that comes with a networked database management system. A smaller embedded database is a better technical solution in this case.

One of the best things about SQLite is that it makes setting up and configuring things a lot easier. It doesn't need a separate server installation, background service management, or complicated deployment steps to work, unlike bigger client-server database systems. This makes it a great choice for an academic full-stack project, where the goal is to show that you made good design and implementation choices instead of copying enterprise-level infrastructure. Using SQLite lets the project stay focused on application logic, validation, data modelling, and persistence without getting sidetracked by unnecessary operational complexity.

SQLite is simple, but it still has the formal database features that a technically sound system needs. It lets you use structured schema design, primary keys, foreign keys, and SQL queries, all of which are important for GapMap's data needs. This means that the project can still show good database reasoning, such as how entities are related to each other, how to keep tasks, timetable entries, and feedback data in a permanent way. Choosing SQLite doesn't mean giving up on rigour; it just means using a tool that is the right size for the project's actual needs.

SQLite is also a good choice for GapMap's development because the app isn't meant to be used by more than one person at a time or to be deployed across multiple locations. The MVP only needs to handle one local user, so it doesn't need the transaction throughput, connection handling, or hosting features that would make a bigger networked DBMS necessary. This is why choosing SQLite shows good scope discipline and keeps the persistence layer from being too complicated.

This choice also helps with maintainability and clarity from a software engineering point of view. It's still easy to set up, look at, and test the database during development, which is helpful when you're changing the schema design or fixing data-related problems. This is especially helpful for a project like GapMap, where the quality of recommendations depends on how consistent and accurate the stored task and timetable data is.

6.4 Security and privacy design

The MVP doesn't have multi-user authentication, but privacy and data integrity are still important. Validation should happen on both the client and the server. Data storage shouldn't keep personal information that isn't needed. If the project is released later, you need to be very careful with how you handle repository secrets, environment variables, and persistent discs. The design also knows that recommendation systems need reliable data, so validation is a privacy and integrity issue, not just a usability issue.

6.5 Accessibility and responsiveness requirements

Accessibility and responsiveness are important non-functional requirements for GapMap. This is because the app's usefulness depends not only on its core features, but also on whether those features stay clear, readable, and usable in real-

world situations. The interface must work with a wide range of common screen sizes while still providing a consistent and easy-to-understand user experience because it is a browser-based productivity system meant for daily use.

The application should be easy to read and use on most laptop screens, and it should also work well on smaller tablet-sized screens. This doesn't mean that the system has to work differently on each device. The main requirement is that the same core features stay available without causing confusion, too many visuals, or a broken page structure. People will probably only use GapMap for short periods of time between other tasks, so the interface should be easy to understand and use quickly on both large and small screens.

Using clear and meaningful labels is an important part of making things accessible. Users shouldn't have to guess what buttons, form fields, page headings, and navigation items do. You should stay away from vague language, especially in a system where quick interaction is important. Clear labelling makes things easier for everyone and is also necessary for accessibility because it makes interactions more predictable and works better with assistive technologies.

The interface should also have a strong colour contrast between text, backgrounds, and things that can be clicked on. This is important to make sure that people can read it easily in normal use and to make it easier for people with vision problems or less than ideal viewing conditions to use. Also, visual grouping shouldn't just depend on small differences in colour. When sections need to be different from each other, the design should also use spacing, borders, headings, and layout structure to make sure the meaning is still clear even if people see colours differently. This backs up the idea that colour shouldn't be the only way to get information.

In GapMap, responsive design should focus less on hiding features and more on keeping the hierarchy and flow of reading. When screen space gets tight, the goal is not to take away important content, but to move it around in a way that makes sense and is easy to use. For instance, the width of a page might need to shrink, multi-column layouts might need to change into stacked sections, and forms might need to be rearranged vertically. Also, the spacing and grouping of the cards should change so that the content is still easy to scan without getting too crowded. These changes should keep the interface's meaning and importance, not just squish all the content into a smaller space.

This requirement is especially important because GapMap needs users to be able to quickly understand task details, timetable data, and recommendation explanations. If responsiveness is not handled well, screens may become visually dense, the order

in which things should be read may become unclear, and it may be harder to find important actions. A well-designed responsive layout should avoid these problems by keeping the structure, emphasis, and readability the same across different viewport sizes.

6.6 Maintainability and code organisation

From the start of the project, maintainability should be a design requirement, not something to think about after the implementation is done. This is very important in GapMap because the system has many layers that work together, such as the user interface, API routes, validation logic, recommendation logic, and persistent storage. If these worries aren't organised clearly from the start, the codebase might be hard to understand, add to, and fix. A structure that can be maintained directly affects the project's technical quality and long-term credibility.

The best way for GapMap to work is to have a clear separation between the client folder and the server folder. This shows how the app is built on a client-server model and helps keep the front end and back end tasks separate. By separating these areas, developers can quickly see where presentation logic ends and where backend processing begins, making the project easier to navigate. It also helps with cleaner deployment and testing by keeping concerns separate in the codebase.

Separating models, controllers, routes, and services into separate layers makes the backend easier to maintain. Routes should list all the API endpoints that are available and connect incoming requests to the right controller functions. Controllers should take care of processing requests, coordinating validation, and building responses. Models should show how persistent entities are structured and make it easy to interact with the database. Services should include the application's more specialised business logic, especially the recommendation engine and any other processing functions that help it. This separation is helpful because it keeps each file and module focused on one task, which lowers the chance of having big, hard-to-manage code units.

This backend structure is very important for GapMap because the recommendation logic is what makes the project stand out the most. By putting that logic in a separate service layer, it's easier to test, improve, and explain without having to deal with routing and request-handling code. It also makes it easier to read because reviewers can find the main decision-making logic without having to look through backend files that don't have anything to do with it. This fits with the project's goal of showing not

only working code, but also good reasoning about how software should be structured.

Separating pages, components, services, styles, and utilities on the front end helps with maintainability. There should be pages for the main views of the system, like the Dashboard, Tasks, Timetable, and Recommendations. Components should be things like cards, forms, navigation items, and alerts that can be used again and again. Frontend services should talk to the backend API, and utilities can have shared helper functions. Styles should be kept in one place instead of being spread out over files that aren't related to each other. This will help keep the look consistent and cut down on duplication. This structure lets the interface grow in a way that is easy to understand and control, and it also lets different parts of the application use the same code.

A well-organised codebase also makes the code easier to read, which is an important quality to think about when grading a computing project. Code that is easy to read is easier to fix, easier to maintain, and easier for teachers or reviewers to use as proof of technical skill. This is possible because of things like consistent naming, a logical folder structure, and a clear separation of concerns. These traits are especially important for GapMap because the project is meant to show a professional and thoughtful approach to full-stack development, not just make a working prototype.

Layer	Representative files / modules	Responsibility
Front end pages	Home, Dashboard, Timetable, Tasks, Recommendation	Top level screens and user tasks
Front end components	NavBar, TaskForm, TimetableForm, GapCard, RecommendationCard	Reusable UI parts
Front end service	api.js	All client-server communication
Back-end routes	taskRoutes, timetableRoutes, recommendationRoutes	HTTP endpoint definitions
Back-end controllers	taskController, timetableController, recommendationController	Validation and request coordination
Back-end services	recommendationEngine	Rules-based business logic
Database	schema.sql, database.js	Persistence and initialisation

EVALUATION AND TESTING PLAN

7.1 Evaluation goals

It is important to look at GapMap's design not only in terms of how well it works, but also in terms of how well it solves the main problem that was found at the start of the project. If a system is hard to use, gives bad advice, or makes interactions more difficult, it may technically work as intended but not provide any real value. Because of this, evaluation must look at both whether the system works and whether it works in a way that is useful, believable, and in line with the project's user-centred goals.

The main goal of GapMap is to help people make better use of their free time by linking task data with schedule data and giving them realistic suggestions. So, the evaluation process should check to see if this goal is really met in real life. The system needs to do more than just store tasks, show timetable sessions, or do maths. The more important question is whether people think the app really helps them figure out what to do when they have a few minutes of free time. This means that the evaluation needs to look at more than just technical performance. It also needs to look at usability, trust, clarity, and how well the solution fits the problem.

One important goal of the evaluation is to find out if users can add tasks and schedule sessions without getting confused. The system needs to make data entry easy and clear because the recommendation process relies on accurate input. The application won't be as useful if users have trouble understanding forms, don't know what fields are required, or aren't sure how to use the main pages. So, when you evaluate, you should think about whether the interface makes it easy to enter and manage task and schedule information with little or no mistakes or hesitation.

A second important goal is to find out if the suggestions seem real and helpful. This is very important for GapMap's success because the recommendation feature is what makes the project stand out. Even if the logic behind a recommendation is sound, it may not work if it seems unrealistic, doesn't fit with the time available, or is hard to trust. Users should think that the suggested tasks fit in with their schedule and workload. So, evaluation should look at more than just whether a recommendation is made. It should also look at whether it seems reasonable, doable, and worth following from the user's point of view.

Another important goal is to make sure that invalid inputs are always blocked. Validation is an important part of both technical quality and user experience because

the system needs structured and reliable data to work properly. Testing should find out if the system blocks bad or malformed inputs correctly, if validation messages are clear, and if the app works as expected when bad data is entered. This helps make sure that the recommendation logic isn't messed up by records that aren't consistent or aren't valid.

The design should also be evaluated in terms of whether the system remains visually clear and low friction. Since GapMap is intended for short, practical interactions between other commitments, the interface must support quick understanding and smooth use. If pages feel cluttered, actions are difficult to locate, or the system requires too many steps to complete simple tasks, then the design would conflict with its own usability aims. Evaluation should therefore consider visual clarity, ease of navigation, readability, and the extent to which the application reduces rather than increases planning effort.

7.2 Formative evaluation

Before decisions about implementation become too set in stone, formative evaluation should happen during the wireframe and prototype stages of the GapMap project. The goal of this evaluation is to find usability problems early on, see if the proposed interface makes its purpose clear, and improve the design based on how users behave. This is an important step in the development process because GapMap is meant to help people make quick and useful decisions. To reach that goal, users need to be able to understand the system easily from the very beginning.

At this point, the evaluation should include a group of students who are representative of the main users of the system. Their involvement is significant as they are most apt to discern whether the app aligns with practical academic routines, disjointed schedules, and deadline-oriented work habits. Instead of just asking for general opinions, the evaluation should be based on short, specific tasks that show how the app is supposed to be used. For example, you could add a task that takes 30 minutes, add a timetable entry, or find out what the app suggests for a 1-hour gap. These activities are directly related to the main functions of GapMap, so they give us useful information about whether the design works in practice.

The main goal of this formative testing is to see if users can understand the interface on their own, without any help or explanation. Specifically, the evaluation should look at whether users understand the names of the pages, whether the structure of the form makes sense, and whether the recommendation output is easy to understand.

Before the final interface is put into use, it is possible to find and fix problems that users have when they are unsure, misinterpret labels, have trouble finding key actions, or don't understand why a recommendation has been made. This is why formative evaluation is so useful: small changes made during the prototype stage can stop bigger usability issues from happening later.

Another benefit of formative evaluation is that it fits with the project's iterative development style. You can use feedback from wireframe and prototype testing to make navigation labels clearer, make forms easier to fill out, improve the layout hierarchy, or make recommendation explanations clearer. In this way, evaluation is not seen as a final decision after development is done, but as something that is always being used to improve the design process. This is in line with both agile development and user-centred design, where feedback is used to gradually improve the system.

Peer testing on a small scale is a good way to evaluate a project of this size. It is quick, cheap, and useful within the limits of the module, and it still gives feedback that is directly related to how the application is meant to be used. This kind of testing isn't very big, but it's enough to find common usability problems, things that are confusing, and flaws in the early design. Because GapMap is a focused productivity tool for one person, this kind of light formative evaluation is appropriate for the project's size and goals.

7.3 Functional and validation test strategy

Test Area	Example test	Expected result
Task creation	Submit valid task with title, duration and future deadline	Task is stored and displayed
Deadline validation	Submit task with deadline earlier than today	Submission is blocked with a clear error
Timetable validation	Submit entry where end time is earlier than start time	Submission is rejected
Recommendation correctness	Provide two sessions with a one-hour gap and tasks of different durations	Only fitting pending tasks are recommended
State persistence	Refresh after creating tasks and timetable entries	Data remains available from the database
Dashboard aggregation	After adding data, open dashboard	Counts match stored task and timetable records

7.4 Usability and accessibility test plan

To see if GapMap's interface is clear, efficient, and right for the situation it will be used in, there should be a structured UI, usability, and accessibility test plan. Because the app is meant to help people make quick decisions when they only have a little time, the quality of the interface is very important to the success of the whole system. So, testing should not only check to see if features are there, but also to see if users can use them easily, confidently, and without any problems.

A good way to test usability would be to watch people do tasks and then ask them a few questions right after. Task-based observation is helpful because it lets the evaluator see how users really move through the system instead of just what they say after they do it. This is especially useful in GapMap, where the goal is to make the user journey as easy and smooth as possible. Watching users try to do realistic things can show hesitation, mistakes in navigation, misunderstandings, or extra complexity that might not be clear just by looking at them. You can then use short questions after a task to get users' first impressions while the interaction is still fresh in their minds.

During testing, the tasks should be similar to the main functions of the MVP. For instance, users might be asked to find the right page for managing their schedule, add a task with a set amount of time, look over a suggestion for a free gap, or explain why a certain suggestion was made. These tasks show directly if the interface supports the intended workflow and if the main functions are clear without any help. Since GapMap is built around a small number of focused actions, usability testing should focus on how quickly and confidently these actions can be done.

Several evaluation criteria are especially relevant. The first is efficiency, which refers to how quickly and directly the user can locate the correct page or function and complete the intended action. In GapMap, efficiency is important because users will probably use the system while they are busy with other things and may not have a lot of time. The second criterion is effectiveness, which means whether the user can finish the task correctly and without making any mistakes. This includes whether people understand forms correctly, whether actions are easy to do, and whether the interface works as expected. The third criterion is satisfaction, which looks at whether the user thinks the interface is easy to use, clear, and low-friction. The project wants to make planning easier, so satisfaction is closely linked to whether the interface feels helpful instead of hard to use.

Accessibility checks should be part of the plan along with usability testing. These tests should look at how the interface works in real life, like how well the colours work together, how easy it is to read the input labels, how big the buttons are, and how well it works on different screen widths. To keep things readable, there needs to be a lot of contrast, especially in areas with a lot of text, like tasks, deadlines, and explanations of recommendations. Input labels should be clear and always linked to fields so that forms are easy to read. Buttons and other interactive parts should be big enough to be easy to use, especially on screens that are the size of a tablet. You should also check the responsiveness to make sure that layouts change clearly without overlapping, crowding, or breaking the flow of reading.

A heuristic review based on Nielsen's ten usability heuristics is another useful tool during the design phase. This is especially true before full implementation is finished, as it lets the proposed interface be tested against established human-computer interaction principles in a systematic way. A review like this can check to see if the system makes the status clear, uses the same words all the time, stops common mistakes, favours recognition over recall, and gives feedback that is easy for users to understand. A heuristic review is especially useful for GapMap because it helps judge the overall quality of the design even if some of the backend features aren't quite finished yet.

This test plan as a whole gives a fair way to judge the interface. Task-based observation shows how users really act, post-task questions get their first impressions, accessibility checks make sure that design considerations are inclusive, and heuristic review gives a structured expert-informed assessment. This combination works for GapMap because it lets students do both practical usability testing and a wider design evaluation, which is what a student project needs.

7.5 Acceptance criteria

Criterion	Measure of success
Core workflow	A user can add timetable data, add tasks, view recommendations, and read dashboard summaries without assistance
Validation quality	Past deadlines and invalid times are blocked inconsistently
Recommendation credibility	Recommendations use only real gaps and provide reasons in plain language
Interface clarity	Representative student users can describe what each page is for after a short walkthrough
Technical completeness	Front-end, back-end and database communicate correctly and persist data

ETHICAL, LEGAL AND PROFESSIONAL CONSIDERATIONS

GapMap is not as risky as systems that handle financial, medical, or other very private personal information, but it still brings up a lot of important ethical, legal, and professional issues. The design document should address these issues because responsible software development is about more than just making things work technically. It also needs to think about how a system might change how users act, how to handle data, and how to present the project in a truthful and professional way.

From an ethical point of view, one important thing is that the app shouldn't make people think that all of their free time is work time. GapMap is meant to help people make decisions when they have short periods of time available, but it shouldn't encourage unhealthy over-scheduling or suggest that every free moment should be used to get more done. This is especially important for students, who may already be under a lot of pressure from their schoolwork. So, recommendations should be given as helpful suggestions instead of orders or demands. The system's goal is to help users make smart decisions, not to force them to finish tasks all the time.

A second ethical issue has to do with how recommendations are made. The app shouldn't trick users into thinking that simple or limited logic is very smart or certain. Because GapMap uses a rules-based recommendation system, it's important that the system is honest about why it makes suggestions. It would be misleading to present weak, approximate, or context-limited logic as if it were authoritative, and this could make users less likely to trust you. This is why adding a clear reason is not only a usability feature, but also an ethical design choice. The system lets users see why a task was suggested, which helps them understand the reasoning behind it and decide if the suggestion is good.

From a legal point of view, the MVP is still pretty limited because it was made mostly for local use. But if the app were to go beyond local development and be used more widely, it would need stricter controls. These would include carefully thinking about how long to keep data, what people expect in terms of privacy, and how safe the repository is. Even simple information about tasks and schedules can show personal habits, deadlines, work patterns, or other information that users would expect to be handled with care. If user accounts, cloud storage, or hosted deployment were to be implemented subsequently, the system would require suitable protections for authentication, secure storage, and regulated access to personal data. At that point,

meeting the relevant data protection standards would be a more important design requirement.

There are also important professional factors to think about when working on the project itself. Good software engineering practices should be used to make the system. These include consistent version control, clear documentation, and a strict approach to project scope. The project should not make its abilities sound better than they are. It is better to say that GapMap is a focused rules-based productivity tool than to make false claims about its intelligence, automation, or predictive power. In an academic setting, this is especially important because the report is part of the graded work and should accurately show what the system does, how it works, and what it can't do.

Professional standards also apply to how the project is run and shown to others. A clear repository structure, a meaningful commit history, design decisions that can be traced, and well-structured documentation all make the final submission more credible. These practices show that the project has been handled in a responsible and organised way, not in a casual or superficial way. They also improve the overall quality of the work by making it easier to understand and judge both the implementation and the reasoning behind it.

CONCLUSION

This design document has shown that GapMap is a software project that is focused, technically sound, and user-centred. The proposed system addresses a clear and relevant problem: many students and other users with similarly fragmented routines have short periods of free time during the day, but they don't have a useful tool to help them figure out which task is best for those gaps. Most productivity tools only let you store tasks or see your schedule, but they rarely do both in a way that makes it easy to make quick and realistic decisions. GapMap sets a clear identity for the project by making timetable-aware recommendations the main focus instead of just managing tasks. This adds a level of technical difficulty that goes beyond a basic CRUD application.

One of the best things about the design is that it stays within its limits on purpose. The project doesn't try to copy all the features of big productivity platforms. Instead, it focuses on solving one problem well. This makes the system easier to understand, easier to put into action, and more believable as an academic computing project. By leaving out higher-risk features like machine learning, syncing with external calendars, and multi-user collaboration, the development team can focus on the main tasks of entering tasks, managing schedules, and making recommendations.

The technical stack of React with Vite, Express, and SQLite that is suggested is also good for the project's goals. It is modern enough to show how to do full-stack development in a way that is realistic and fits the size of the minimum viable product. This combination makes it possible to have a modular client-server architecture, structured persistence, and iterative development without adding extra complexity. Because of this, the project is ready to show that it can use sound technical reasoning and put it into practice.

The paper has also shown that the project can be planned and carried out in a logical and well-supported way. The fact that the project has formal diagrams, structured requirements, defined scope boundaries, risk analysis, technical architecture, and evaluation planning shows that it is not based on vague ideas but on a structured design process. These things show that GapMap can be made, tested, and reviewed using a strict software engineering method. This is important not only for the quality of the implementation, but also for meeting the requirements of the project brief and the higher-band assessment criteria.

Most importantly, the system was made to be improved and changed over time. You can test early ideas before putting them into action by using wireframes and

prototypes. The architectural choices make it easier to develop in modules and find and fix bugs. Validation rules make sure that the quality of user input is good, and the evaluation plan gives you a way to judge usability, recommendation quality, and overall effectiveness. These choices together make a development path where quality is improved by making changes over and over again instead of waiting until the end to fix things.

If done correctly, GapMap should show that it has good technical merit, is original, and understands the link between user-centred design and full-stack software engineering. It is a project with a clear goal, a solid technical base, and a real connection to what users need. For these reasons, it is a computing project that is well-justified, useful, and possible to complete.

REFERENCES

Express (n.d.) Routing. Available at: https://expressjs.com/en/guide/routing.html

Express (n.d.) Using Middleware. Available at: https://expressjs.com/en/guide/using-middleware.html

MDN Web Docs (n.d.) Client-side form validation. Available at: https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation

MDN Web Docs (n.d.) Using the Fetch API. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MDN Web Docs (n.d.) Window: localStorage property. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

Nielsen Norman Group (n.d.) 10 usability heuristics for user interface design. Available at: https://www.nngroup.com/articles/ten-usability-heuristics/

React (n.d.) Conditional rendering. Available at: https://react.dev/learn/conditional-rendering

React (n.d.) useState. Available at: https://react.dev/reference/react/useState

SQLite (n.d.) CREATE TABLE. Available at: https://www.sqlite.org/lang_createtable.html

SQLite (n.d.) Foreign Key Support. Available at: https://www.sqlite.org/foreignkeys.html

Vite (n.d.) Getting started. Available at: https://vite.dev/guide/

W3C Web Accessibility Initiative (n.d.) WCAG 2 Overview. Available at: https://www.w3.org/WAI/standards-guidelines/wcag/

APPENDIX A. FULL FUNCTIONAL REQUIREMENT SET

ID	Requirement	Priority	Design note
FR1	The system shall allow the user to create a task with title, duration, priority, energy level, and optional deadline	Must	Needed for all later recommendation behaviour
FR2	The system shall allow the user to delete a task and update its status	Must	Supports realistic workflow and list maintenance
FR3	The system shall allow the user to create a timetable entry with day, start and end times.	Must	Required for gap detection
FR4	The system shall allow the user to delete a timetable entry	Must	Necessary for correcting plan changes
FR5	The system shall generate recommendations only from real gaps between sessions	Must	Core logic constraint
FR6	The system shall allow the user to define an active day window	Should	Improves relevance of recommendations
FR7	The system shall show at least one explanation for why a task was selected	Should	Supports trust and transparency
FR8	The system shall display a dashboard summary of stored data	Could	Useful overview but depends on core features working already

APPENDIX B. FULL NON-FUNCTIONAL REQUIREMENT SET

Category	Requirement	Indicative target
Usability	Primary actions should be understandable without a manual	New users can complete core tasks after a short walkthrough
Performance	Common GET/POST interactions should feel immediate in local development	Typical response well under a few seconds
Reliability	Stored data must persist after refresh and app restart	SQLite retains valid records
Accessibility	Forms and buttons must remain readable and usable for common laptop/tablet scenarios	Clear labels, contrast, and sensible click target
Maintainability	Code should be modular and separated by responsibility	Pages / components / routes / controllers / models / services structure
Security	Invalid input must not reach storage unchecked	Server-side validation for all task and timetable routes

APPENDIX C. SAMPLE FORMATIVE INTERVIEW QUESTIONS

- When you have a 20–60-minute gap between classes, how do you currently decide what to do?
- What is frustrating about using a normal to-do list for short periods of time?
- Would you trust a system to suggest a task, and what explanation would you need to trust it?
- Which page names feel most obvious to you: Tasks, Timetable, Recommendations, Dashboard?
- What would make the app feel too complicated or too similar to software you already dislike?

APPENDIX D. DIAGRAM SET INCLUDED IN THIS DOCUMENT

- **Figure A. Requirements Funnel**
- **Figure B. Stakeholder power-interest map**
- **Figure C. Competitor positioning matrix**
- **Figure D. Proposed milestone timeline**
- **Figure E. Proposed GitHub workflow**
- **Figure F. Design Risk matrix**
- **Figure G. Reconstructed low-fidelity wireframe**
- **Figure H. Reconstructed medium-fidelity prototype**
- **Figure I. High-Fidelity / Final prototype**
- **Figure J. Information architecture / site map**
- **Figure K. Use case diagram**
- **Figure L. Activity flowchart for recommendation generation**
- **Figure M. Sequence diagram for fetching recommendations**
- **Figure N. GapMap System Architecture**
- **Figure O. Entity relationship diagram**
- **Figure P. Recommendation scoring logic**